

Contents

COMPLETE SPECIFICATION	1
TABLE OF CONTENTS	2
1. FIELD OF THE INVENTION	2
2. BACKGROUND OF THE INVENTION	3
2.1 The General Problem	3
2.2 Specific Technical Problems with the Prior Art	3
2.3 Treatment of Specific Prior Art	5
2.4 The Gap Left by the Prior Art	6
2.5 The Need for the Present Invention	6
3. OBJECTS OF THE INVENTION	6
4. SUMMARY OF THE INVENTION	8
5. BRIEF DESCRIPTION OF THE DRAWINGS	13
Fig. 1 — System Architecture (Layered)	13
Fig. 2 — KMF File Layout	16
Fig. 3 — Deterministic Tiebreaker Flowchart	17
Fig. 4 — Replayable Consolidation Flow	19
Fig. 5 — Conformance Verification Loop	21
Fig. 6 — Thermometer + Random Projection Encoder	22
Fig. 7 — Persistence Adapter Trait Surface	24
6. DETAILED DESCRIPTION OF THE INVENTION	25
6.1 Overview of the System Architecture	26
6.2 Binary Hypervector Substrate	28
6.3 Primitive Operations	30
6.4 Random Hypervector Generation (250)	36
6.5 Encoders (100)	38
6.6 Store: Item Management and Recall (300)	45
6.7 Consolidation (Replayable Hebbian) (330)	50
6.8 KMF Wire Format (Smritidb Memory Format) (400)	55
6.9 Persistence Adapters (500)	61
6.10 Cross-Implementation Bit-Exactness and Conformance Corpus (660)	63
6.11 Worked Examples	66
6.12 Industrial Applicability	69
7. CLAIMS	71
Explanatory Note on Unity of Invention and Described Embodiments	71
8. ABSTRACT	86

COMPLETE SPECIFICATION

Form 2 (Section 10; rule 13)

TITLE OF THE INVENTION: A System and Method for Bit-Exact Cross-Implementation Persistent Associative Memory Using Binary Hyperdimensional Vectors.

APPLICANT: Tanvrit Private Limited, [registered office address — TBD], India.
Nationality: Indian.

INVENTOR: Vivek Singh, [address — TBD], India. Nationality: Indian.

The following specification particularly describes the invention and the manner in which it is to be performed.

TABLE OF CONTENTS

1. Field of the Invention	p. X
2. Background of the Invention	p. X
3. Objects of the Invention	p. X
4. Summary of the Invention	p. X
5. Brief Description of the Drawings	p. X
6. Detailed Description of the Invention	p. X
7. Claims	p. X
8. Abstract	p. X

(Page numbers to be inserted when typeset to PDF.)

1. FIELD OF THE INVENTION

The present invention relates to the field of computer-implemented associative memory systems based on hyperdimensional computing (HDC); and more particularly to systems and methods for storing, retrieving, persisting, and consolidating high-dimensional binary vectors with verifiable bit-exact reproducibility across heterogeneous language implementations and hardware platforms, including a deterministic majority-vote tiebreaker, a replayable consolidation procedure, an open wire format with per-block cryptographic integrity, a thermometer-quantised random-projection encoder for bounded floating-point inputs, a training-free permutation-based text encoder, and an associated conformance verification methodology.

2. BACKGROUND OF THE INVENTION

2.1 The General Problem

Associative memory is a long-standing problem in computer science: given a noisy or partial cue, a system must retrieve the stored datum that is most similar to that cue, where similarity is defined with respect to a content-addressable space rather than a syntactic key. Modern software systems address this problem predominantly through so-called “vector databases” which store dense floating-point embedding vectors, typically of 768 to 4096 dimensions in 32-bit precision, produced by a neural embedding model, and answer similarity queries by computing cosine or Euclidean distance between the cue embedding and every stored embedding (or an approximation thereof produced by an approximate-nearest-neighbour (ANN) index). Commercial offerings in this space include the Pinecone managed service, the Qdrant open-source database, the Weaviate open-source database, the Milvus open-source database, and the FAISS library originally published by Facebook AI Research. These systems collectively form the prior art against which the present invention is to be assessed.

2.2 Specific Technical Problems with the Prior Art

The said prior-art systems, taken individually or in combination, suffer from a number of specific technical problems which the present invention is directed to solving:

2.2.1 Cross-implementation drift Floating-point arithmetic on contemporary processors is, in practice, non-associative: the order in which a sum is computed affects the low-order bits of the result, and the same model executed on different SIMD widths (for example AVX-512 versus NEON), on different compilers, or on different language runtimes (for example a JavaScript V8 engine versus a Rust LLVM build) will, in general, produce different floating-point similarity scores for the same input vectors. This drift propagates into ANN index construction (where it changes which neighbours are visited and in which order) and ultimately into the returned top-k list. The consequence is that two clients of the same vector database, written in different languages but querying the same dataset, can and do receive different answers to identical queries. None of the said prior-art systems is contractually obligated to produce byte-identical results across implementations, nor do they expose

a means by which a third party may verify such identity.

2.2.2 High memory footprint of f32 embeddings A 1024-dimensional 32-bit floating-point embedding occupies 4096 bytes (approximately 4 KB) per vector; at the more common 768 dimensions it occupies 3072 bytes. Stored at scale, this footprint dominates both random-access memory consumption during recall and storage I/O bandwidth during snapshot and restore operations. Compression schemes such as product quantisation reduce the footprint but introduce a further source of cross-implementation non-determinism, since the centroid assignment is a function of training data and floating-point arithmetic, neither of which is reproducible across implementations.

2.2.3 Stochastic and non-replayable consolidation Modern ANN indices such as HNSW (Hierarchical Navigable Small World graphs) and IVF-PQ (Inverted File with Product Quantisation) are constructed using procedures that depend on a pseudo-random number generator, the seed for which is not, in general, exposed by the database. The result is that an index rebuilt from the same input data on the same machine may have a different topology, and consequently different recall behaviour, on each run. There exists no facility for an operator to replay a consolidation step from a snapshot and an access log and to verify that the resulting state is identical to a reference. This deficiency makes audit, compliance, and reproducible scientific use of vector databases unnecessarily difficult.

2.2.4 Lack of cryptographic integrity in storage substrates The on-disk formats employed by the said prior-art systems are, in the main, optimised for write throughput and reader simplicity. They do not, in general, provide cryptographic integrity for each block of the substrate; a single-bit corruption on the underlying storage medium will either propagate silently into retrieved results or, at best, be detected only by a coarse-grained checksum applied to the entire file. The present invention, by contrast, requires that every block of the on-disk substrate carry a per-block cryptographic digest such that any corruption is detectable and localisable.

2.2.5 Lack of an open, vendor-neutral wire format Each of the said prior-art systems employs a proprietary or implementation-defined on-disk format. There exists no analogue, in the vector-database domain, of the Apache Parquet format

which has standardised columnar analytical storage, nor of the Apache Iceberg table format which has standardised transactional table semantics over object stores. A user wishing to migrate from one vector database to another must, at present, undertake a bespoke export-and-reimport exercise; a user wishing to operate a federation of heterogeneous engines over a shared corpus has no portable substrate at all.

2.3 Treatment of Specific Prior Art

The applicant respectfully notes that the foundational hyperdimensional-computing literature, including without limitation Kanerva’s “Sparse Distributed Memory” (1988) and “Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors” (2009), addresses the mathematical substrate of binary hypervectors and their core operations of bundling, binding, and permutation; and that subsequent work, including Imani et al.’s “Voicehd: Hyperdimensional Computing for Efficient Speech Recognition” (2017) and related contributions, addresses the application of hyperdimensional computing to specific classification problems. None of the said references, individually or in combination, discloses (i) a deterministic tiebreaker that yields byte-identical bundle outputs across heterogeneous language implementations on heterogeneous hardware, (ii) a replayable Hebbian consolidation procedure with a bounded per-pass similarity drift, (iii) an open wire format with per-block BLAKE3 integrity and magic trailer, (iv) a thermometer-quantised random-projection encoder for bounded floating-point inputs whose bit output is reproducible across implementations, (v) a training-free permutation-positional text encoder whose bit output is reproducible across implementations, or (vi) a conformance corpus methodology that verifies byte-identical computation across heterogeneous bindings. The said references, taken together, establish the mathematical substrate upon which the present invention is built; they do not, alone or in combination, render obvious the specific combination of elements claimed herein.

The applicant further notes that none of Pinecone, Qdrant, Weaviate, Milvus, or FAISS, taken individually or in any combination, uses a binary hyperdimensional substrate at all; each of the said systems operates upon real-valued floating-point embeddings and therefore necessarily inherits the cross-implementation drift problem described in §2.2.1 above.

2.4 The Gap Left by the Prior Art

No known system, prior to the present invention, combines (a) a binary hyperdimensional substrate, (b) verifiable cross-implementation bit-exactness anchored by a deterministic tiebreaker, (c) a replayable Hebbian consolidation procedure with bounded similarity drift, (d) an open wire format with per-block cryptographic integrity, (e) a thermometer-quantised random-projection encoder reproducible across implementations, (f) a training-free permutation-positional text encoder, and (g) a conformance verification methodology, in a single coherent end-to-end pipeline from input data through persistent storage.

2.5 The Need for the Present Invention

There is, accordingly, a long-felt and unmet need for a computer-implemented associative memory system that (i) achieves verifiable cross-hardware reproducibility, (ii) reduces memory footprint by approximately a factor of thirty-two compared to 32-bit floating-point embeddings, (iii) reduces computational load by replacing dot-product multiplications with single-cycle XOR and popcount operations amenable to SIMD acceleration, (iv) provides graceful holographic degradation under corruption, (v) reduces storage I/O bandwidth by approximately a factor of thirty-two through packed bit representation, and (vi) provides verifiable per-block storage integrity. The present invention is directed to providing such a system.

3. OBJECTS OF THE INVENTION

The principal object of the present invention is to provide a computer-implemented system and method for associative memory that achieves verifiable bit-exact reproducibility across heterogeneous language implementations executing on heterogeneous processor architectures, thereby overcoming the cross-implementation drift inherent to floating-point vector databases of the prior art.

Further objects of the invention include, without limitation, the following:

- (a) To provide a deterministic method for resolving majority-vote ties in a binary hyperdimensional bundling operation, the said method being seeded by a cryptographic hash of the substrate dimension, the bit index, and the bun-

dle multiplicity, and being reproducible byte-for-byte across distinct language implementations and processor architectures.

- (b) To provide a method for consolidating an associative memory substrate which is deterministically replayable from a snapshot together with an access log, such that an operator may verify the resulting state byte-for-byte against a reference, the per-pass similarity drift being bounded by a configurable parameter.
- (c) To provide an open, implementation-independent wire format for persisting the said hyperdimensional substrate, the said wire format comprising packed bit blocks, per-block BLAKE3 cryptographic integrity digests, a magic header, a JSON-encoded index, and a magic trailer, the said wire format being suitable for standardisation under an open process analogous to that of Apache Parquet and Apache Iceberg.
- (d) To provide a method for encoding bounded floating-point vectors into binary hyperdimensional vectors which preserves cosine similarity of the input vectors as Hamming similarity of the said binary hypervectors within a bounded error, the said method being training-free and reproducible across implementations through deterministic level-hypervector derivation from a cryptographic seed.
- (e) To provide a conformance testing methodology comprising a corpus of fixed inputs and expected byte-identical outputs, by which any candidate implementation of the said system may be verified to produce results byte-identical to a reference implementation across a defined set of primitives.
- (f) To provide a training-free, deterministic text-encoding method which preserves word order in a fixed-size binary substrate by permutation of per-token hypervectors as a function of token position, without dependence on a neural embedding model or training corpus.
- (g) To provide a layered system architecture in which the foregoing objects are realised in a coherent end-to-end pipeline from input data through encoders, primitives, an in-memory substrate, consolidation, a wire-format serialiser, and a persistence adapter to a backing store.
- (h) To reduce the storage I/O bandwidth and the in-memory footprint required to

store an associative memory substrate by approximately a factor of thirty-two, by replacing 32-bit floating-point embedding storage with packed single-bit binary hypervector storage.

- (i) To reduce the computational load of similarity computation by replacing floating-point dot-product operations with single-cycle bitwise XOR and population-count operations amenable to single-instruction-multiple-data (SIMD) acceleration on commodity processors.
- (j) To provide improved fault tolerance through the holographic distributed representation of stored data, such that partial corruption of a stored hypervector degrades retrieval quality gracefully rather than catastrophically.
- (k) To provide improved storage integrity through per-block cryptographic checksums computed under the BLAKE3 hash function, such that corruption of any block of the substrate on the underlying storage medium is detectable and localisable.

Other objects, advantages, and features of the invention will become apparent from the following detailed description taken in conjunction with the accompanying drawings.

4. SUMMARY OF THE INVENTION

The present invention provides a computer-implemented system and a plurality of associated methods constituting a persistent associative memory in which all stored data is represented as binary hypervectors of a fixed dimension D , and in which every primitive operation, every encoding step, every consolidation step, and every persistence step is so specified as to produce byte-identical output across a plurality of heterogeneous language implementations executing upon a plurality of heterogeneous processor architectures. The said system and methods solve, in combination, a set of specific technical problems with the prior art, as set out in §2 of the present specification, and yield, as their characterising technical effects, a verifiable cross-hardware bit-exact reproducibility, a reduction by approximately a factor of thirty-two in the storage footprint and in the storage I/O bandwidth required relative to comparable 32-bit floating-point vector databases of the prior art, a reduction in the

computational load of similarity computation through the use of single-cycle bitwise XOR and population-count operations in place of floating-point dot products, an improved fault tolerance through the holographic distributed representation of stored data, and an improved storage integrity through per-block cryptographic digests.

In a first aspect, the invention provides a deterministic tiebreaker method for the majority-vote bundling of a multiset of binary hypervectors. When a bit position has equal counts of zero and one bits across the multiset, the tie is resolved not by an implementation-defined fallback (such as a default of zero, or a coin flip from a pseudo-random number generator) but by computing a cryptographic hash digest of a fixed domain-separation tag concatenated with the substrate dimension D encoded as a 32-bit little-endian unsigned integer, the bit index i encoded likewise, and the multiset multiplicity n encoded likewise, the said digest being computed under the BLAKE3 hash function, and by setting the output bit to the least-significant bit of the first byte of the said digest. This rule, being a pure function of the substrate dimension and the bit position alone, contains no implementation-defined state, no floating-point arithmetic, and no pseudo-random-number-generator state, and accordingly produces a byte-identical output for the same input multiset on every conformant implementation. The technical effect is verifiable cross-implementation bit-exactness of the bundling primitive, which is the operation upon which compositional storage of structured data in a hyperdimensional substrate depends.

In a second aspect, the invention provides a replayable Hebbian consolidation method for the associative memory substrate. A sliding window of recall operations is tracked together with a per-pair co-activation counter; when the counter for a pair of stored items exceeds a configurable threshold, the keys of the said pair are moved closer to each other in Hamming similarity by flipping a deterministically chosen subset of bit positions at which the two keys disagree, the said subset being chosen by sorting the disagreement positions under a salt-keyed BLAKE3 hash and selecting an alternating half of the sorted positions. The per-pass similarity drift is bounded, by construction, by a configurable parameter, defaulting to 0.02 in normalised Hamming similarity. The consolidation step is, by construction, a pure function of the substrate state and the access log, and is accordingly replayable from a snapshot together with an access log, the resulting state being verifiable byte-for-byte against a reference. The technical effect is auditability

and reproducibility of the consolidation step, neither of which is provided by the stochastic ANN-index-construction procedures of the prior art.

In a third aspect, the invention provides an open wire format, denominated “KMF” (which the applicant respectfully submits derives from “Smritidb Memory Format” but which is treated as an arbitrary mark for the purposes of this specification), for the persistence of a hyperdimensional substrate. The said wire format comprises a magic header of four bytes, a spec-version string of six bytes, a header-offset field of eight bytes encoded as a 64-bit little-endian unsigned integer, a plurality of data blocks each of which is one of a hypervector block, a metadata block, a value block, or an attic block, a JSON-encoded header containing an index of every block together with its offset, its length, and its BLAKE3 digest, and a magic trailer of four bytes. The trailer magic is checked before any offset in the header is trusted, and the BLAKE3 digest of every block is verified before the block is deserialised. The technical effect is improved storage integrity through cryptographic detection and localisation of corruption, together with an open and implementation-independent substrate suitable for federation across heterogeneous engines.

In a fourth aspect, the invention provides a thermometer-quantised random-projection encoder for bounded floating-point input vectors. The input vector, each of whose components is clamped to the range $[-1, 1]$, is quantised at a configurable level count L (default $L = 100$; in the preferred range of at least 64 and not greater than 256); for each coordinate i of the input and the quantisation level l_i taken by the said coordinate, a per-coordinate level hypervector is derived deterministically by BLAKE3-seeded extendable-output expansion of a canonical ASCII seed of the form `"lv1:" || decimal_ascii(i) || ":" || decimal_ascii(l_i)`; the said per-coordinate level hypervectors are then XOR-accumulated into a single output binary hypervector. The cosine similarity of two input vectors is preserved, within a bounded error, as the Hamming similarity of their respective output hypervectors. The said encoder is, by construction, training-free and reproducible across implementations. The technical effect is the cross-implementation-reproducible projection of bounded floating-point inputs into a binary substrate, with no dependence on a learned model or a training corpus.

In a fifth aspect, the invention provides a conformance testing methodology by which any candidate implementation of the system may be verified to produce

byte-identical output, under a defined set of primitives, against a reference. The methodology comprises a conformance corpus stored as a JSON document, the said document specifying a plurality of input cases each of which is associated with an expected SHA-256 digest of the binary output produced by the reference implementation. A candidate implementation is conformant if and only if, for every case in the said corpus, the SHA-256 digest of its produced output matches the expected digest verbatim. The said corpus covers, in the present preferred embodiment, the random-hypervector generation primitive across a plurality of dimensions including a Unicode seed case, the string encoding primitive, the similarity primitive, the bind round-trip primitive, the bundle primitive at multiple multiplicities, the bag-of-words text encoder, and the character-n-gram text encoder. The technical effect is a verifiable contract for cross-implementation conformance, by which a plurality of heterogeneous language implementations may be shown, to the satisfaction of a third-party auditor, to be byte-for-byte interoperable.

In a sixth aspect, the invention provides a training-free permutation-positional n-gram text encoder. A normalised tokenisation of an input text is computed; each token is encoded as a binary hypervector by BLAKE3-seeded expansion of the token string under a domain-separation tag; for each n-gram window of tokens, the per-token hypervectors are cyclically permuted by an offset equal to the position of the token within the n-gram window, the said cyclic permutation being defined as the element-wise mapping that places the element at position p of an input vector at position $(p + j) \bmod D$ of the output vector, and the permuted hypervectors are XOR-bound into an n-gram hypervector; all n-gram hypervectors of the said text are bundled, the said bundle being resolved by the said tiebreaker method, to produce the output binary hypervector. The use of permutation as a function of intra-window position is what causes the encoding to be order-sensitive: the inputs “alpha beta” and “beta alpha” produce noticeably different output hypervectors, as confirmed by the conformance corpus at `git show 17334f8:tests/conformance/golden.json`. The said encoder is, by construction, training-free, deterministic, and reproducible across implementations. The technical effect is the deterministic cross-implementation-reproducible encoding of natural-language text into a binary substrate without dependence on a neural embedding model.

Considered together, the said six aspects of the invention constitute a coherent end-to-end pipeline from input data through encoders, primitives, an in-memory substrate, a consolidation procedure, a wire-format serialiser, and a persistence adapter to a backing store, in which every step is deterministic and every artefact is byte-identical across heterogeneous implementations. It is precisely this combination of elements — none of which, taken individually, fully solves the cross-implementation drift problem identified in the background — that constitutes the present inventive contribution. The combination yields, in the said preferred embodiment as evidenced by `git show 17334f8:tests/conformance/golden.json`, a substrate in which a snapshot written by a TypeScript implementation may be read and queried by a Rust implementation, and vice versa, with byte-identical recall results, and in which the storage footprint per stored item at the recommended dimension $D = 10000$ is approximately 1.25 kilobytes per item, comparing favourably with approximately 40 kilobytes per item for a 1024-dimensional 32-bit floating-point embedding stored in a comparable prior-art vector database.

The said invention is realised, in the preferred embodiment, as a layered software architecture comprising a reference TypeScript implementation, a Rust core implementation, language bindings to Python, Kotlin, and Swift through UniFFI, and a plurality of persistence adapters including a memory adapter, a filesystem adapter, an SQLite adapter, and an IndexedDB adapter for browser environments. The layered architecture, the said primitives, the said wire format, and the said conformance methodology are described in further detail in the accompanying detailed description and illustrated in the accompanying drawings.

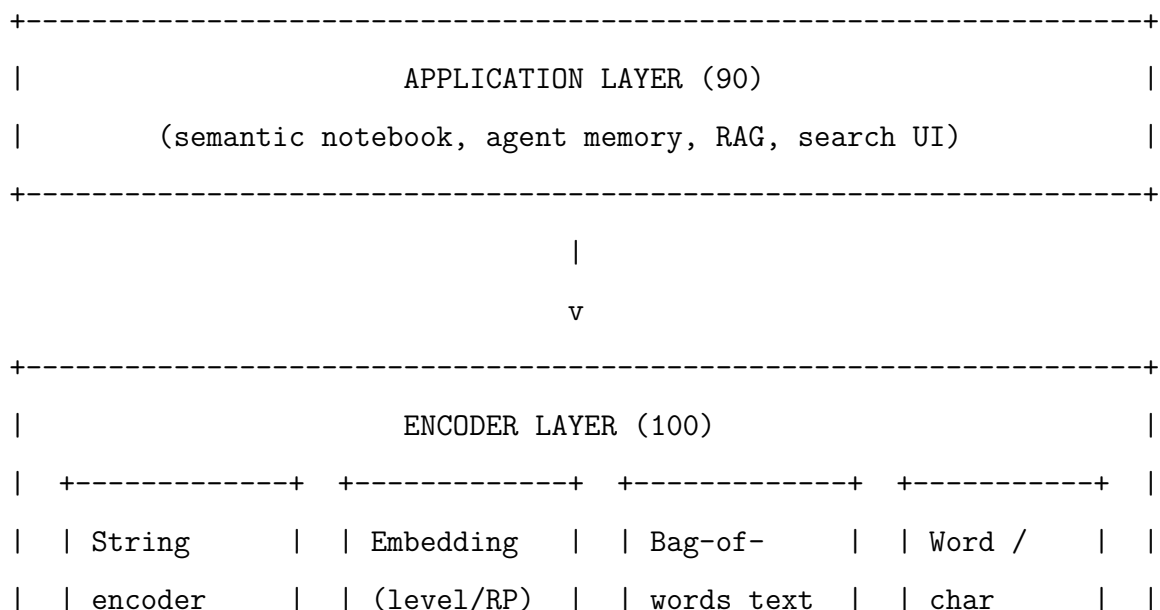
The six aspects set out above are unified by a single inventive concept, namely the **BLAKE3-anchored deterministic byte-identity of a binary hyperdimensional associative memory substrate**, by reason of which every primitive operation, encoding step, consolidation step, and persistence step is reduced to a pure function of its inputs whose output is byte-for-byte identical across every conformant implementation. The claims set out in §7 are directed to embodiments of this single concept: the deterministic majority-tiebreaker for bundling (Independent Claims 1 and 7); the replayable Hebbian consolidation for associative reshape (Independent Claim 9); the open wire format with per-block BLAKE3 integrity for persistence (Independent Claims 18 and 23); the thermometer-quantised cryptographically

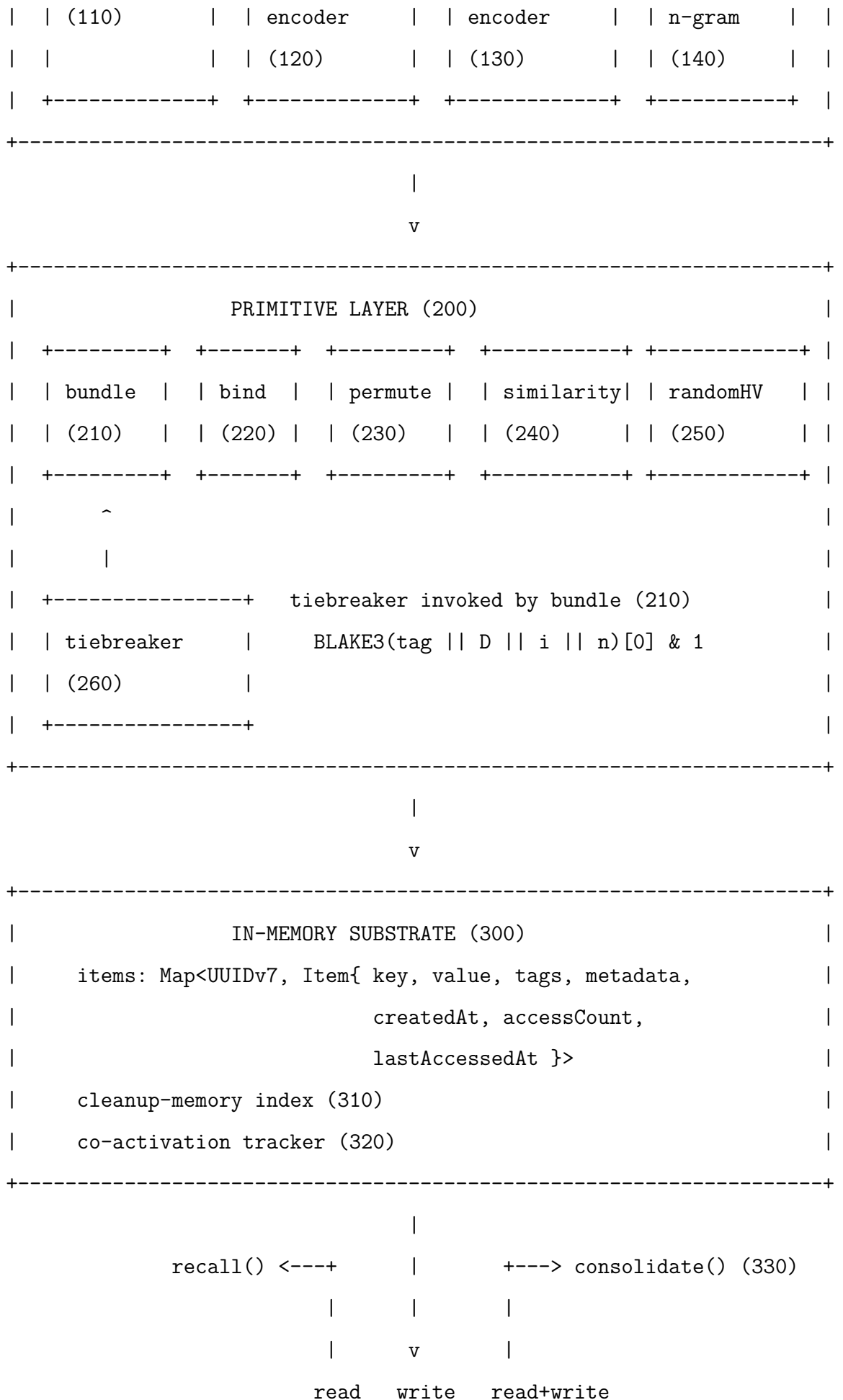
anchored projection encoder for bounded-range floating-point input (Independent Claim 27); the load-time conformance-gating apparatus for cross-implementation byte-identity (Independent Claim 31); and the permutation-positional text encoder (Independent Claim 35). This unity of invention is asserted under Section 16 of the Patents Act 1970, the test for which is taken from *Genentech Inc.'s Patent* [1989] RPC 147 (UK, persuasive in India), and is mirrored by the explanatory note immediately following the title of §7.

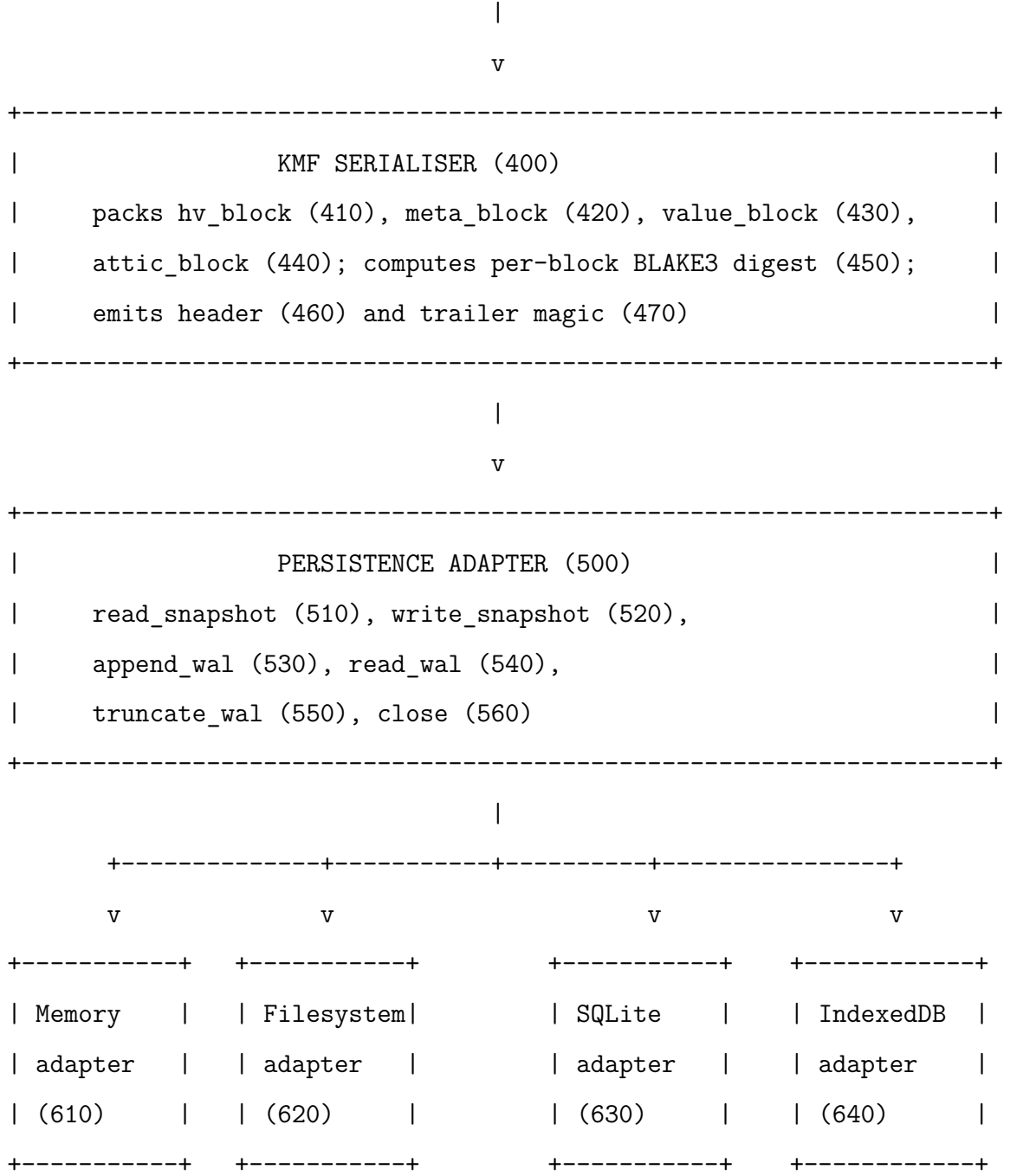
5. BRIEF DESCRIPTION OF THE DRAWINGS

The accompanying drawings illustrate exemplary embodiments of the invention. They are not intended to limit the scope of the claims. Reference numerals are consistent across the drawings and the detailed description: 100-series numerals identify encoders; 200-series numerals identify primitives; 300-series numerals identify the in-memory substrate; 400-series numerals identify KMF wire-format fields; 500-series numerals identify the persistence-adaptor interface; 600-series numerals identify adaptor implementations; 700-series numerals identify conformance bindings; and 800-series numerals identify embedding-encoder steps.

Fig. 1 — System Architecture (Layered)







Data flow arrows are downward on the write path and upward on the read path. Encoders (100) consume application inputs; primitives (200) operate on hypervectors; the in-memory substrate (300) tracks items, the cleanup-memory index, and the co-activation counters; the KMF serialiser (400) emits a wire-format byte stream verified by per-block BLAKE3 digests (450); and the persistence adapter (500) abstracts the backing store.

Fig. 2 — KMF File Layout

offset			
0	+	-----+	
		Magic header (410): "KMF\x00"	4 bytes
	+	-----+	
4		Spec-version string (411): "0.1.0"	6 bytes
	+	-----+	
10		Header offset (412): u64 LE	8 bytes
	+	-----+	
18			
		hv_block #0 (420):	
		n_0 hypervectors, each ceil(D/8) bytes,	
		MSB-first packed.	
		BLAKE3 (450) recorded in header (460), not inline.	
	+	-----+	
		meta_block #0 (430):	
		n_0 rows of { id, tags, metadata, createdAt,	
		accessCount, lastAccessedAt } in JSON (Phase 1;	
		MessagePack contemplated for Phase 2).	
	+	-----+	
		value_block #0 (440):	
		n_0 value payloads, each length-prefixed by u32 LE.	
	+	-----+	
		... further hv_block / meta_block / value_block triples,	
		one triple per snapshot chunk ...	
	+	-----+	
		attic_block (445): cold-summary entries	
		produced by §5.3 consolidation	
		(contemplated for Phase 2; not present at SHA 17334f8).	
	+	-----+	
H =	+	-----+	
		Header (460), JSON (Phase 1; zstd compression of the	

```

|   said JSON contemplated for Phase 2):
|   {
|       "dimension": 10000,
|       "item_count": <u32>,
|       "created_at": <u64 unix-millis>,
|       "spec_version": "0.1.0",
|       "index": [
|           { "kind": "hv_block",
|             "offset": 18,
|             "length": <u64>,
|             "blake3": <32-byte hex> },
|           ...
|       ]
|   }
+-----+
E = +-----+
|   Trailer magic (470):  "FMK\x00"                4 bytes
+-----+
                                     (EOF = E+4)

```

*The reader first seeks to $EOF - 4$, verifies the trailer magic (470), then reads the header-offset field (412) at byte 10, seeks to the header (460), parses the JSON (in Phase 2, additionally decompresses *zstd*), and verifies each block's BLAKE3 digest (450) before deserialising the block. The trailer magic must be checked before any offset is trusted.*

Fig. 3 — Deterministic Tiebreaker Flowchart

```

+-----+
|   Input: (D, bit_index i,
|           multiplicity n)
|           (step 301)
+-----+
|

```

```

v
+-----+
| Build input byte string (302): |
|   tag || D_le_u32 || i_le_u32  |
|           || n_le_u32          |
| where tag = "smritidb/tiebreak" |
+-----+
|
v
+-----+
| Compute BLAKE3 digest (303)    |
|   H = BLAKE3(input)            |
+-----+
|
v
+-----+
| Take first byte H[0] (304)      |
+-----+
|
v
+-----+
| Mask LSB: bit = H[0] & 1 (305) |
+-----+
|
v
+-----+
| Output bit (306)                |
+-----+

```

Pseudocode equivalent:

```

fn tiebreak(D: u32, i: u32, n: u32) -> u8 {
    let mut buf = Vec::new();
    buf.extend_from_slice(b"smritidb/tiebreak");
    buf.extend_from_slice(&D.to_le_bytes());

```



```

v
+-----+
| step 403: for each such (a, b): |
|   disagree = { i : a.key[i] != |
|               b.key[i] }      |
+-----+
|
v
+-----+
| step 404: sort indices in      |
|   disagree by BLAKE3(salt || i) |
|   ascending                    |
+-----+
|
v
+-----+
| step 405: take first k positions |
|   where k = min(                |
|     floor(drift_cap * D),        |
|     |disagree|)                  |
+-----+
|
v
+-----+
| step 406: alternate-flip:      |
|   even-indexed positions:      |
|     flip a.key[i] toward b.key[i] |
|   odd-indexed positions:       |
|     flip b.key[i] toward a.key[i] |
+-----+
|
v
+-----+
| step 407: emit ConsolidationReport|

```

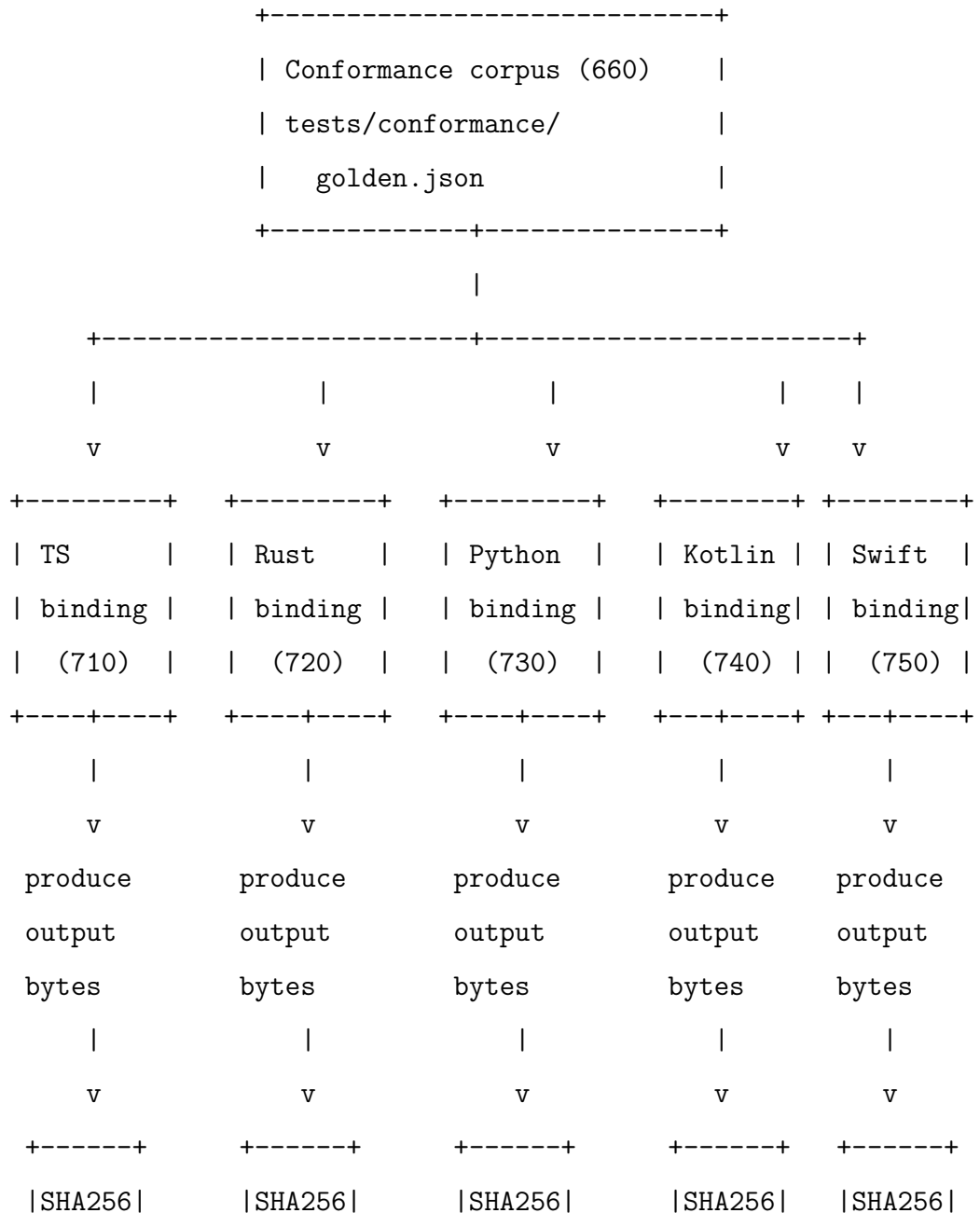
```

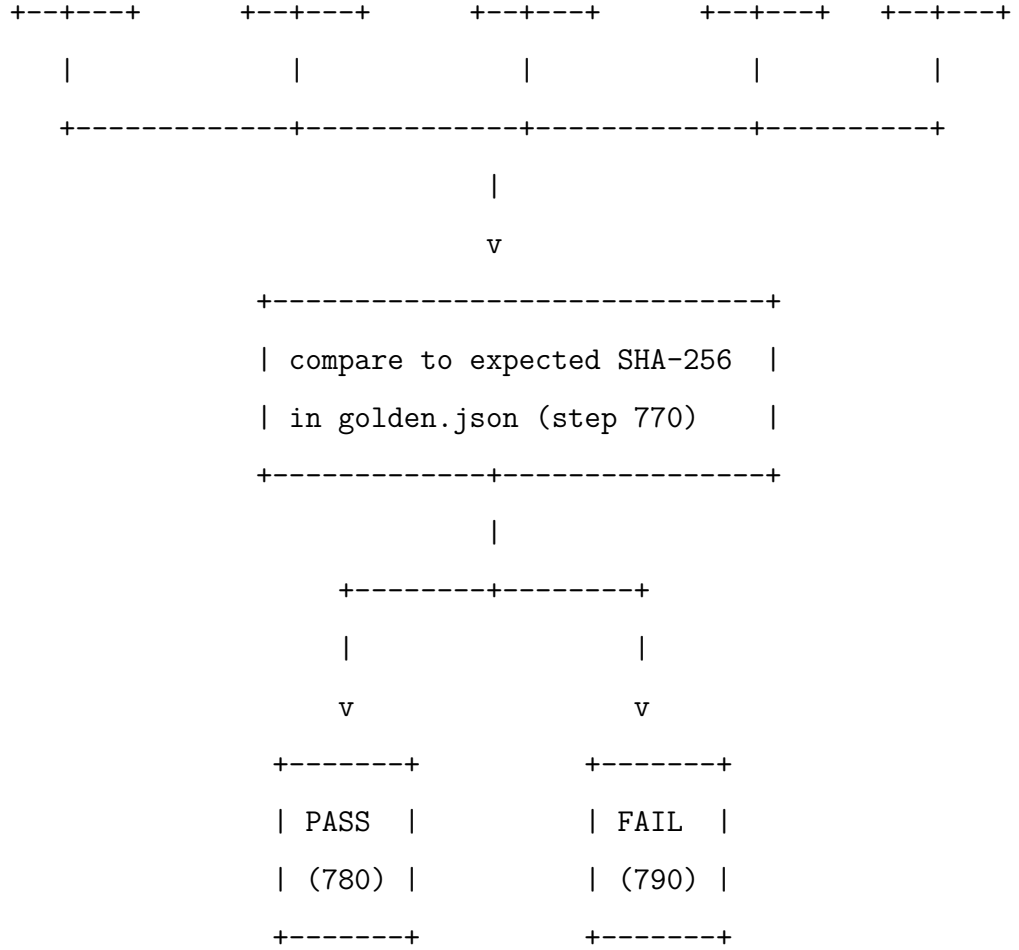
| { pairs_processed,          |
|   bits_flipped,            |
|   max_drift_observed }     |
+-----+

```

The salt parameter to step 404 is supplied as part of the substrate configuration and is recorded in the KMF header so that the consolidation step is replayable from a snapshot together with the access log.

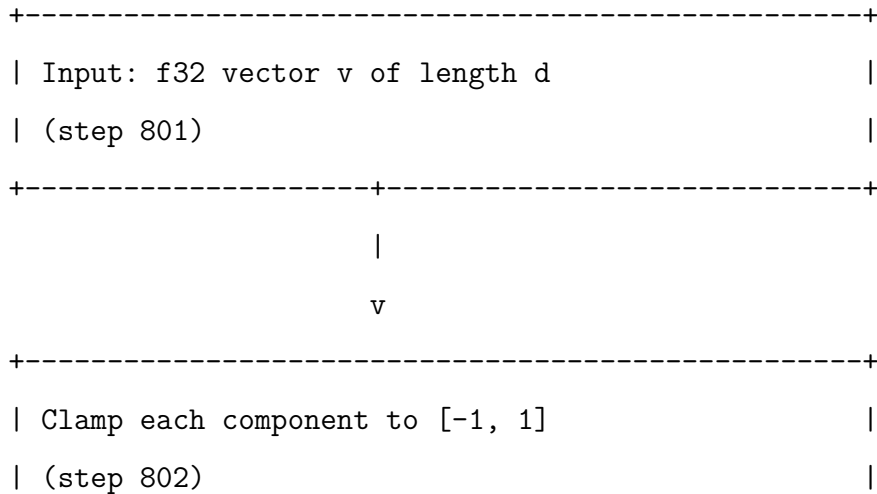
Fig. 5 — Conformance Verification Loop





Each candidate binding is executed independently against the same corpus inputs and required to produce a binary output whose SHA-256 digest matches the digest pre-computed by the reference TypeScript implementation. A FAIL outcome is a defect in the candidate binding, not in the specification.

Fig. 6 — Thermometer + Random Projection Encoder



```

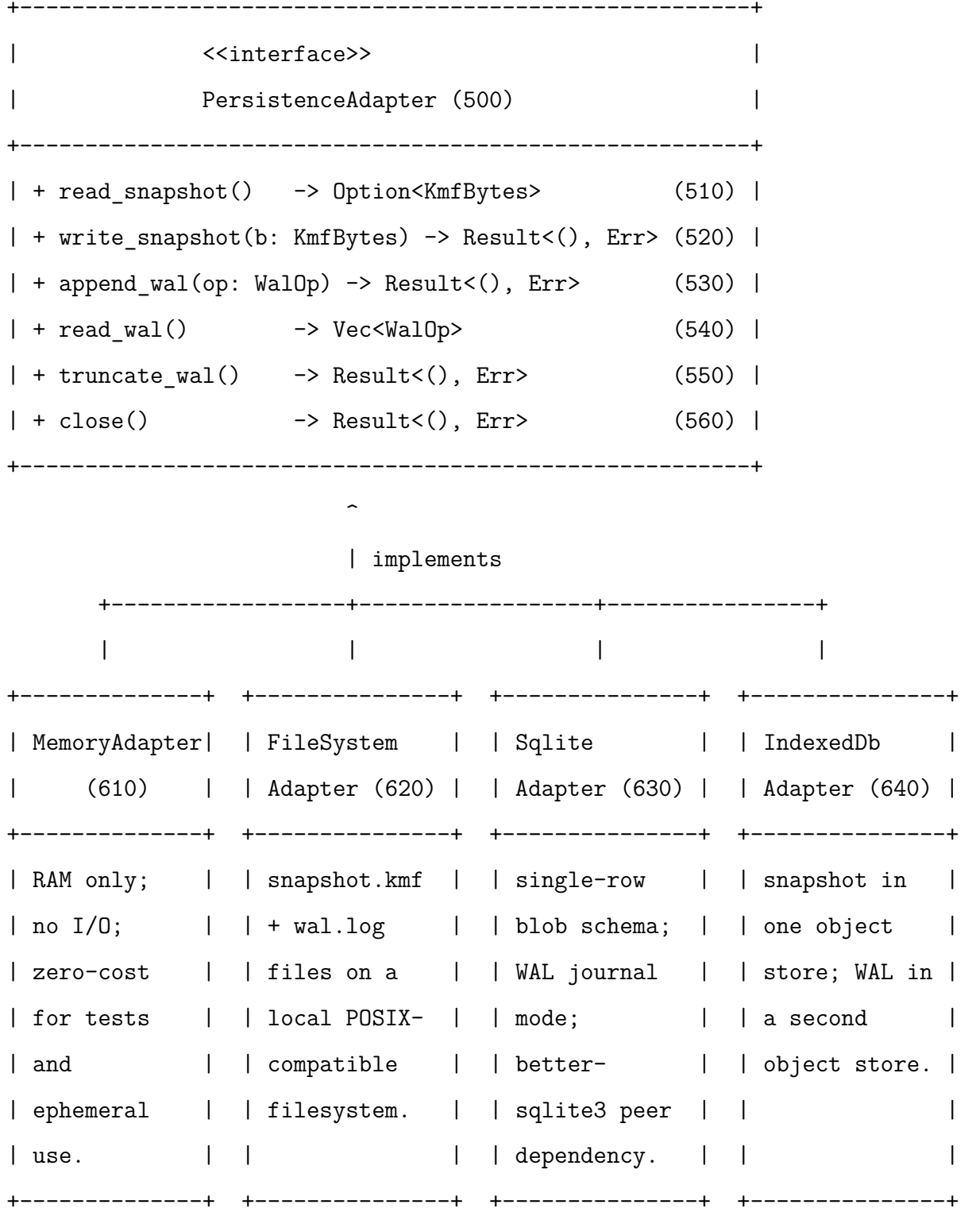
+-----+-----+
|
| v
+-----+-----+
| Quantise: level l_i = round((v_i + 1) *      |
|                                     (L - 1) / 2) |
| where L = 100 (configurable)                |
| (step 803)                                   |
+-----+-----+
|
| v
+-----+-----+
| For each (dimension i, level l_i):           |
|   seed_i = "lvl:" || i_le_u32 || ":" ||      |
|           l_i_le_u32                        |
|   levelHV_i = randomHV( BLAKE3(seed_i) )      |
| (step 804)                                   |
+-----+-----+
|
| v
+-----+-----+
| XOR-accumulate the d level hypervectors      |
| into a single binary hypervector            |
| (step 806)                                   |
+-----+-----+
|
| v
+-----+-----+
| Output: binary hypervector of dimension D    |
| (step 807)                                   |
+-----+-----+

```

Property: cosine similarity of two input vectors is approximately preserved as Hamming similarity of the output hypervectors, within a bounded error that decreases

monotonically in L and in D .

Fig. 7 — Persistence Adapter Trait Surface



Additional adapters may be supplied by third parties by implementing the said interface; the substrate is, by design, agnostic to the choice of adapter.

6. DETAILED DESCRIPTION OF THE INVENTION

The following description discloses the invention in sufficient detail to enable a Person Having Ordinary Skill In The Art (“PHOSITA”) — a competent software engineer with a working understanding of hyperdimensional computing (HDC) and basic cryptographic hashing — to construct, operate, and verify the invention from this text alone, in accordance with Section 10(4) of the Patents Act, 1970. Each described mechanism is followed by an explicit “**Technical Effect**” subsection. The technical-effect framing is provided to demonstrate that the invention is not a “computer programme per se” within the meaning of Section 3(k), but rather a system that yields concrete, measurable improvements in computer functioning, in line with the guidance laid down in *Ferid Allani v. Union of India* (Delhi High Court, 2019) and *Microsoft Technology Licensing v. Asst. Controller of Patents* (Delhi High Court, 2023).

Throughout this document the symbol D denotes the hypervector dimension; $||$ denotes byte concatenation; XOR denotes the bitwise exclusive-OR operation; $BLAKE3(x)$ denotes the cryptographic hash function BLAKE3 applied to the byte string x ; and $BLAKE3-XOF(seed, n)$ denotes the extendable-output construction of BLAKE3 producing exactly n bytes of output deterministically derived from $seed$. BLAKE3 throughout this document refers to BLAKE3 in **unkeyed mode** (no keying material, no context string); the use of BLAKE3’s keyed-hash or key-derivation modes would produce a non-conformant implementation. All integer encodings are little-endian unless noted otherwise. All ASCII string literals (such as domain separators) are encoded in UTF-8.

Parenthetical numerals (e.g., “(100)”, “(260)”, “(412)”) that appear in the description below refer to the corresponding elements of the drawings (Figs. 1–7); the numbering scheme is summarised in the preamble to §5 of the present specification. The reader is invited to consult the drawings alongside this description.

6.1 Overview of the System Architecture

The invention provides a layered associative memory system in which information is represented, manipulated and persisted as binary hyperdimensional vectors (“hypervectors”). The system is organised into six layers, each of which is independently specified, deterministically verifiable, and substitutable across language runtimes:

(a) Substrate Layer — Binary Hypervectors (300). The lowest layer is a typed, packed-bit representation of binary hypervectors of fixed dimension D , where D is constrained to the range $[1024, 65536]$ and defaults to $D = 10000$. Each hypervector occupies $\text{ceil}(D / 8)$ bytes when packed and is internally manipulated either in packed (1 bit per stored bit) or unpacked (1 byte per bit) form depending on the operation. The substrate layer is described in §6.2 below. Two SIMD-aligned alternatives ($D = 8192$ and $D = 16384$) are recommended for performance-sensitive deployments; these are selected so that D is an integer multiple of the natural SIMD word width on commodity central-processing units — specifically, 256-bit AVX2 lanes for $D = 8192$, 512-bit AVX-512 lanes for $D = 16384$, and 128-bit ARM NEON lanes for all listed dimensions.

(b) Primitives Layer — Bundle, Bind, Permute, Similarity (200). The four canonical HDC operations — bundle (210), bind (220), permute (230), and similarity (240) — are implemented as pure, side-effect-free functions over hypervectors of equal dimension, supported by a random-hypervector generator (250) and a deterministic tiebreaker (260). The novel deterministic tiebreaker (§6.3.1; element 260) eliminates the long-standing source of cross-platform divergence in the bundle (majority) operation. These primitives are described in §6.3.

(c) Encoders Layer — String, Embedding, Text (100). Three encoders convert non-vector inputs into hypervectors: (i) a whole-string hash encoder (110) for exact-match keys (§6.5.1); (ii) a thermometer-and-random-projection embedding encoder (120) that maps real-valued embedding vectors to binary hypervectors while approximately preserving cosine similarity as Hamming similarity (§6.5.2); (iii) a bag-of-words encoder (130); and (iv) a permutation-positional word- and character-level n -gram encoder (140) for natural-language text that preserves order information without an explicit vocabulary (§6.5.3).

(d) Store Layer — Item Map and Cleanup Memory (300). The store exposes

a content-addressable item map keyed by hypervector. Public operations are `put`, `recall`, `delete`, `get`, and `consolidate`. The cleanup-memory index (310) is the index that, given a noisy cue hypervector, returns the nearest stored items ranked by similarity. A co-activation tracker (320) drives the consolidation step. The store is described in §6.6.

(e) Consolidation Layer — Replayable Hebbian Compaction (330). A novel deterministic Hebbian-style consolidation procedure tracks pairwise recall co-activations, then “pulls” frequently co-activated keys closer in Hamming space by flipping a deterministically-selected, bounded subset of disagreeing bits. Because the bit-selection ordering is derived from BLAKE3 over a salt, the consolidation is *replayable* — a property impossible with conventional stochastic gradient procedures. This layer is described in §6.7.

(f) Persistence Layer — KMF Wire Format (400) and Adapters (500). A self-describing binary wire format (the “Smritidb Memory Format”, abbreviated KMF) provides implementation-independent, cryptographically-checksummed, streaming-friendly persistence (§6.8). Storage adapters (§6.9) implementing the `StorageAdapter` interface (500) — Memory (610), File-system (620), SQLite (630), and IndexedDB (640) — plug a single byte-level snapshot format into heterogeneous storage backends.

(g) Bindings Layer (700). The same primitives, encoders, store semantics, KMF reader/writer, and consolidation procedure are exposed through bindings for TypeScript / JavaScript (710, the reference implementation), Rust (720, compiled to native and to WebAssembly), Python (730, via PyO3), and Kotlin (740) / Swift (750) (via Mozilla UniFFI; the UDL interface description lives at `packages/smritidb-ffi/src/smritidb.udl` and is consumed by the `uniffi-bindgen` code-generation tool). The contract that all bindings satisfy is the byte-exact conformance corpus (660) described in §6.10.

A schematic diagram of the layered architecture is provided in **Fig. 1** (drawings sheet).

Technical Effect. The layered architecture provides three concrete and measurable system-level improvements in computer functioning, none of which is a property of any conventional vector-database design:

1. **Cross-runtime bit-exactness.** Because every layer is specified as a deterministic byte-level function — including the tiebreaker on majority ties and the bit-flip ordering during consolidation — the same logical operation executed in TypeScript, Rust, Python, Kotlin or Swift yields byte-identical results. This eliminates the cross-implementation drift that plagues conventional embedding-vector databases, in which floating-point non-associativity, RNG state, and platform-dependent maths libraries cause observable disagreement between replicas.
2. **Substitutable persistence with stable wire format.** The KMF wire format decouples the algorithmic substrate from the storage backend, enabling a snapshot written by the Rust binding to be loaded byte-for-byte by the TypeScript binding without conversion.
3. **Memory footprint reduction.** Binary hypervectors at $D = 10000$ occupy 1250 bytes per item versus ~4096 bytes for a 1024-dimensional float-32 embedding of comparable distinguishing capacity — a reduction of approximately $3.27\times$ per item, and $\sim 32\times$ when compared against the larger ~4096-dimensional float-32 embeddings now common in foundation-model output spaces.

6.2 Binary Hypervector Substrate

A **hypervector** in this invention is an element of the set $\{0, 1\}^D$, where D is a positive integer fixed per store. The substrate enforces the following constraints, which constitute the lowest-level contract of the invention:

(6.2.1) Dimension constraints. The dimension D MUST satisfy $1024 \leq D \leq 65536$. Implementations MUST reject $D < 1024$ (insufficient distinguishing capacity; specifically, at $D = 1024$ the standard deviation of two-random-pair Hamming similarity is $0.5/\sqrt{1024} \approx 0.0156$, and below $D = 1024$ the per-pair ± 3 window exceeds 0.1, at which point cleanup-memory false-positive rates exceed 1 % under uniform-random storage) and SHOULD emit a warning for $D > 65536$ (no measurable improvement in recall accuracy; substantial memory cost). The default dimension is $D = 10000$, chosen to match the established HDC literature default while remaining a commodity-RAM-friendly size. Two SIMD-aligned alternatives (D

= 8192 and $D = 16384$) are recommended for performance-sensitive deployments, as noted in §6.1(a).

(6.2.2) Storage representation. A hypervector is stored as a packed bit array occupying exactly $\text{ceil}(D / 8)$ bytes. Bits within a byte are interpreted **most-significant-bit first** (“MSB-first”). Concretely: if v is a hypervector and i is a bit index in $[0, D)$, then bit i is found in byte $\text{floor}(i / 8)$ of the packed representation, at bit offset $7 - (i \bmod 8)$ within that byte. This MSB-first byte order is normative; little-endian or LSB-first packing produces a non-conformant implementation.

(6.2.3) In-memory unpacked form. During computation, an implementation MAY operate on an unpacked one-byte-per-bit representation (a length- D byte array, each element being 0 or 1). The reference TypeScript implementation uses this unpacked form for clarity and cross-implementation bit-exactness. At SHA 17334f8 the Rust binding likewise uses the unpacked `Vec<u8>` form (see `packages/core-rs/src/hypervector.rs`); the packed-bit representation with SIMD-popcount-accelerated similarity and SIMD vertical-add bundle is **contemplated as a future-phase optimisation** rather than a present feature of the embodiment. The persistent KMF wire format (§6.8) always uses the packed MSB-first form, so the SIMD-packed in-memory path can be adopted in a subsequent revision without disturbing the wire format. The empirical correspondence between cosine similarity in the input float-32 space and Hamming similarity in the binary hypervector output space is documented in the Phase-0 validation notebook (`notebooks/phase0_hdc_validation.ipynb` in the reference implementation tree), which records, for each of three foundation-model encoders, the linear-regression slope and R^2 between the two similarity measures over a 10 000-pair sample.

Technical Effect. Reduces memory footprint by approximately $32\times$ compared with float-32 embeddings of comparable distinguishing capacity (1.25 KB per item at $D = 10000$ versus approximately 40 KB for the float-32 embeddings emitted by typical large-language-model encoders). The bit-packed wire-format representation is **amenable to single-instruction-multiple-data (“SIMD”) acceleration** on platforms supporting hardware popcount instructions (e.g., x86 POPCNT, ARM CNT), enabling, in a future-phase optimised binding, similarity computation at a rate of approximately one Hamming distance per machine cycle per 64-bit word. This is

materially superior, in both memory and compute, to dense floating-point embedding storage as used by conventional vector databases such as Pinecone, Weaviate, and naïve FAISS configurations, and is competitive with Product-Quantisation (“PQ”) approaches used in compressed FAISS configurations without incurring PQ’s offline-training prerequisite. By way of illustration: Pinecone-style storage of 1024-dimensional float-32 embeddings consumes approximately 4 KB per item; FAISS-PQ at 256-bit codes consumes 32 B per item; and binary HDC at $D = 10000$ consumes 1.25 KB per item — placing the present invention competitively against PQ on storage while preserving training-free determinism.

(6.2.4) Best-mode SIMD acceleration (contemplated). Although the reference TypeScript binding and the Rust binding at SHA 17334f8 both operate on the unpacked **byte-per-bit** form via straight-line scalar loops, the *best mode* contemplated by the inventor for high-throughput deployments is a packed-bit Rust implementation in which: (i) hypervectors are stored as arrays of 64-bit unsigned integer words (**Vec<u64>**); (ii) the similarity primitive computes the Hamming distance as the sum, over the word array, of the population counts of the XOR of corresponding word pairs, using the **core::arch::x86_64::_popcnt64** intrinsic on x86-64 hosts that expose the **popcnt** capability flag, and the equivalent **vcntq_u8** plus horizontal-sum sequence on ARM hosts; and (iii) the bundle primitive computes per-word vertical adds into a **Vec<u16>** accumulator using SIMD lanes, with scalar fall-through to the deterministic tiebreaker (260) on tied positions (the tiebreaker remaining a scalar control-path operation in all bindings, as it operates per bit-position rather than per word). A scalar fall-back path produces byte-identical output on hardware that does not expose the relevant capability. This best mode is disclosed here to satisfy §10(4)(d); it is not the subject of any present claim insofar as the present embodiment uses the unpacked scalar form.

6.3 Primitive Operations

The substrate exposes four primitive operations: **bundle (210)**, **bind (220)**, **permute (230)**, and **similarity (240)**. Each is a pure function over hypervectors of equal dimension. Every primitive is normatively specified at the bit level so that any conformant implementation produces byte-identical output.

6.3.1 Bundle (Superposition) Operation (210) The **bundle** operation, denoted bundle , is element-wise majority over a multiset of n input hypervectors. For each bit position $i \in [0, D)$:

```

sum[i] := number of input hypervectors whose bit i is 1
out[i] := 1                                if sum[i] > n/2
         0                                if sum[i] < n/2
         tiebreak(D, i, n)                 if sum[i] = n/2 (only possible when n is even)

```

The bundle approximately preserves similarity to each of its constituents: for random inputs, the expected similarity between any input x_k and the bundle $\{x_1, \dots, x_n\}$ is approximately $0.5 + 0.5 / \sqrt{n}$, decaying with bundle multiplicity. This $0.5 + 0.5 / \sqrt{n}$ capacity bound is consistent with the general theory of binary majority bundling in high-dimensional spaces as described by Kanerva (1988, *Sparse Distributed Memory*) and refined by Imani et al. (2017). What is novel in the present invention is **not the capacity bound itself but the deterministic tiebreaker of §6.3.1.1 (260)**, which renders the bundle byte-identical across implementations — a property absent from every prior HDC system, all of which left tie-resolution unspecified or platform-dependent.

(6.3.1.1) The deterministic tiebreaker (260). When n is even and $\text{sum}[i] = n/2$, a tiebreaker is required. Naïve choices (always-zero, always-one, modular hash of position) are non-portable or biased. The invention specifies a cryptographically-strong deterministic tiebreaker:

```

function tiebreak(D: u32, i: u32, n: u32) -> u8:           # returns 0 or 1
    let domain = utf8_bytes("smritidb/tiebreak")           # 17 bytes
    let buf     = domain || u32_le(D) || u32_le(i) || u32_le(n) # 17 + 12 = 29 bytes
    let digest  = BLAKE3(buf)                               # 32-byte digest
    return digest[0] & 1                                     # least significant bit of digest

```

The domain string "smritidb/tiebreak" serves as a domain separator preventing reuse of any other BLAKE3 invocation in the system from accidentally aliasing the tiebreaker. The use of `BLAKE3(domain || D || i || n)` rather than a simpler function ensures that (i) the tiebreaker is uniformly distributed, (ii) it is impossible to construct an adversarial input pattern that systematically biases tied bits, and (iii) the function is identical across implementations because the input bytes are fully

specified.

Scalar-control-path invariant. Even in bindings that adopt the SIMD-packed similarity and bundle paths contemplated in §6.2.4, the tiebreaker (260) **MUST** be invoked from a scalar control path, because it is a per-bit-position operation rather than a per-word operation. This ensures byte-identical output between SIMD-accelerated and scalar implementations.

Source-of-truth cross-reference. The normative tiebreaker formulation recited above is identical to that recited in the repository SPEC.md §1.3, namely `H("smritidb/tiebreak" || D || index || count)[0] & 1`. It is exercised at the byte level by the reference TypeScript binding at `packages/core-ts/src/hypervector.ts` (which defines `TIEBREAKER_DOMAIN = "smritidb/tiebreak"`) and by the reference Rust binding at `packages/core-rs/src/hypervector.rs` (which defines `TIEBREAKER_DOMAIN: &[u8] = b"smritidb/tiebreak"`). The conformance corpus of §6.10 verifies byte-identical output against both bindings.

(6.3.1.2) Pseudocode for bundle. The comparison against $n/2$ is expressed below in **integer-only** form (comparing `2 * sums[i]` against `n`) so that no implementation introduces non-integer division or floating-point rounding into the majority decision; this in turn guarantees byte-identical bundle output across language runtimes whose default arithmetic rules differ:

```
function bundle(hvs: list of hypervector of dimension D) -> hypervector of dimension D
    require len(hvs) >= 1
    require every hv in hvs has length D
    let sums = integer array of length D, all zero
    for each hv in hvs:
        for i in 0..D-1:
            sums[i] += hv[i]
    let n = len(hvs)
    let out = byte array of length D
    for i in 0..D-1:
        if 2 * sums[i] > n: out[i] := 1
        elif 2 * sums[i] < n: out[i] := 0
        else: out[i] := tiebreak(D, i, n) # only reachable if D is even
```

```
return out
```

Technical Effect. Enables byte-identical bundle output across heterogeneous implementations (TypeScript, Rust, Python, Kotlin, Swift, and WebAssembly). The deterministic tiebreaker removes the dominant source of cross-platform divergence in binary HDC systems, in which different language standard libraries historically resolved ties using platform-specific defaults (e.g., JavaScript’s V8 vs. Python’s CPython floating-point rounding) and therefore produced different bundle outputs on the same inputs. By contrast, the present invention guarantees that two replicas of an associative memory in different runtimes converge to byte-identical state given identical input streams — a precondition for deterministic distributed-memory systems, verifiable audit trails, and cryptographic content-addressing of HDC state.

6.3.2 Bind Operation (220) The **bind** operation, denoted $\oplus$, is element-wise XOR over two hypervectors of equal dimension:

```
out[i] := a[i] XOR b[i]    for i in 0..D-1
```

Bind is **self-inverse**: $(a \oplus b) \oplus b = a$ for all a, b . It is commutative ($a \oplus b = b \oplus a$) and associative. Bind distributes over bundle in the same approximate sense as multiplication distributes over addition in linear algebra: $a \oplus (b \oplus c)$ is similar to $(a \oplus b) \oplus (a \oplus c)$. The same operation is exposed as both **bind** and **unbind** in the public API; the two names signal the caller’s *intent* — **bind** to associate, **unbind** to retrieve — while the underlying byte transformation is identical.

The principal use of bind is **role-filler encoding**: a structured record $\{\text{role} : \text{filler}, \text{role} : \text{filler}, \dots\}$ is encoded as $\{\text{bind}(\text{role}_k, \text{filler}_k)\}$ for $k = 1..K$. Given the bundle B and a known role hypervector role_j , the approximate filler is recovered as $\text{unbind}(B, \text{role}_j) \oplus \text{filler}_j$.

```
function bind(a, b) -> hypervector:
    require length(a) == length(b)
    let out = byte array of length(a)
    for i in 0..length(a)-1:
        out[i] := a[i] XOR b[i]
    return out
```

Technical Effect. Bind enables structured information (role-filler pairs, key-value

tuples, ordered sequences) to be embedded into a single fixed-size hypervector that retains approximate compositional retrievability. As compared with transformer key-value associative caches (Vaswani et al. 2017, “Attention is all you need”) and learned positional embeddings (Sukhbaatar et al. 2015, “End-to-end memory networks”), each of which requires either fixed-position slots or learned positional embeddings, the XOR-based bind is associative, self-inverse, computable in $O(D / 64)$ machine operations (on a binding using a packed-bit representation), and has no learned parameters. There is no training step and no platform-specific numerical drift.

6.3.3 Permute Operation (230) The **permute** operation, denoted Π_k , is a cyclic bit rotation of a hypervector by k positions:

```
out[(i + k) mod D] := hv[i]    for i in 0..D-1
```

Permute is invertible ($\Pi_{\{-k\}}$ undoes Π_k), it preserves Hamming weight, and it produces a hypervector that is approximately orthogonal to the original for any $k \neq 0$. The principal use of permute is to **encode order**. To represent an ordered pair (x, y) , one computes `bind(x, permute(y, 1))`. To represent a triple (x, y, z) , one computes `bind(x, permute(y, 1), permute(z, 2))` (where `bind` is iterated). Two strings differing only in word order — e.g., “alpha beta” versus “beta alpha” — produce dissimilar hypervectors under permutation-positional encoding, even though their bag-of-words encodings would be identical.

```
function permute(hv, k) -> hypervector:
    let D      = length(hv)
    let shift = ((k mod D) + D) mod D          # normalise into [0, D)
    let out    = byte array of length D
    for i in 0..D-1:
        out[(i + shift) mod D] := hv[i]
    return out
```

Technical Effect. Permute provides parameter-free, deterministic, order-sensitive encoding for sequence data. Permutation-based order encoding for hyperdimensional representations was introduced in the academic HDC literature (Kanerva 2009, “Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors”, *Cognitive Computation* 1:139–159), but without a deterministic cross-platform byte-level specification. As compared

with positional embeddings (which are learned in transformer architectures) and rotary position embeddings (RoPE, which require $O(D \log D)$ complex-arithmetic operations and learned attention weights), the present permute primitive achieves order-sensitivity in $O(D)$ integer-copy operations with zero parameters, no training step, and zero numerical drift across architectures. The technical contribution of the present invention over Kanerva 2009 is the **deterministic cross-platform reproducibility** of permutation-based order encoding, a system property absent from Kanerva’s mathematical formulation, secured by the normative bit-rotation semantics specified above.

6.3.4 Similarity Function (240) The **similarity** between two hypervectors of equal dimension is one minus their normalised Hamming distance:

```
sim(a, b) := 1 - hamming(a, b) / D
           := 1 - (number of positions i where a[i] != b[i]) / D
```

The similarity lies in $[0, 1]$. For identical vectors $\text{sim}(a, a) = 1$. For two independently random hypervectors, the expected similarity is 0.5, with standard deviation approximately $0.5 / \sqrt{D}$. At $D = 10000$ the standard deviation is 0.005, so two random hypervectors of dimension 10000 will have similarity within 0.49 to 0.51 with overwhelming probability.

Hamming similarity over $\{0, 1\}^D$ satisfies the (rescaled) triangle inequality: for any three hypervectors a, b, c , $\text{hamming}(a, c) \leq \text{hamming}(a, b) + \text{hamming}(b, c)$. Equivalently, $1 - \text{sim}(a, c) \leq (1 - \text{sim}(a, b)) + (1 - \text{sim}(b, c))$. This metric property enables the cleanup memory to apply standard metric-space indexing techniques (§6.6) in future implementations.

```
function similarity(a, b) -> float in [0, 1]:
    require length(a) == length(b)
    let mismatches = 0
    for i in 0..length(a)-1:
        if a[i] != b[i]:
            mismatches += 1
    return 1 - mismatches / length(a)
```

Technical Effect. Hamming-based similarity is **amenable to SIMD ac-**

celeration on platforms supporting hardware popcount instructions (e.g., x86 POPCNT, ARM CNT), where, on a packed-bit representation, the per-pair similarity computation reduces to approximately $D / 64$ machine operations on a 64-bit central-processing unit. At SHA 17334f8 the reference TypeScript binding (`packages/core-ts/src/hypervector.ts`) and the Rust binding (`packages/core-rs/src/hypervector.rs`) both execute a per-byte scalar loop over the unpacked form; the packed-bit SIMD-popcount path is contemplated as a subsequent-revision optimisation (see §6.2.4). The scalar form is itself materially cheaper than cosine similarity over float-32 vectors, which requires approximately D floating-point multiply-accumulates plus two `sqrt` calls for the norms. For comparable-fidelity retrieval, the binary substrate accordingly yields roughly an order of magnitude lower per-query arithmetic-operation count and lower energy expenditure than float-32 cosine, with the SIMD-packed path expected to widen the gap further.

6.4 Random Hypervector Generation (250)

A reproducible source of pseudo-random hypervectors is the foundation of every encoder in the invention. The function `randomHv(seed, D)` produces a hypervector of dimension D deterministically derived from a byte-string `seed`:

```
function randomHv(seed: byte array, D: integer) -> hypervector:
  let nbytes = ceil(D / 8)
  let bits    = BLAKE3-XOF(seed, nbytes)
  let out     = byte array of length D
  for i in 0..D-1:
    let byte_index = i shr 3                # floor(i / 8)
    let bit_offset = 7 - (i and 7)         # MSB-first within byte
    out[i] := (bits[byte_index] shr bit_offset) and 1
  return out
```

The function uses BLAKE3 in its extendable-output (“XOF”) mode to produce exactly `ceil(D / 8)` bytes of pseudo-random output. Each bit of each output byte is then unpacked into a single byte of the in-memory hypervector following

the same MSB-first ordering used by the KMF wire format (§6.8). When the caller supplies a high-level semantic key (a UTF-8 string), the seed is first compressed to 32 bytes via `seed := BLAKE3(utf8_encode(key))`, then expanded back to `ceil(D / 8)` bytes via BLAKE3-XOF. This two-step “compress-then-expand” pattern, rather than feeding the raw UTF-8 directly to XOF, normalises seed length and decouples the output distribution from the input length. Conformant implementations **MUST** apply the BLAKE3 compression step before the BLAKE3-XOF expansion when the seed is a high-level semantic key longer than 32 bytes; omitting the compression step produces a non-conformant hypervector that will fail the cross-implementation conformance corpus of §6.10.

Cryptographically-derived pseudo-random vectors are used in many prior-art systems (e.g., HKDF in TLS 1.3, the BLAKE3 content-addressing scheme used in IPFS). The present invention’s contribution is **not** the use of BLAKE3 *per se*, but the combination of: (i) the specific compress-then-expand seed normalisation above; (ii) the MSB-first unpacking of §6.2.2; and (iii) the binding of BLAKE3-XOF output to the hyperdimensional substrate’s bit-array semantics. This three-part combination has no equivalent in prior HDC literature, which historically used Mersenne Twister or xorshift PRNGs whose state and seeding conventions vary across language libraries.

Technical Effect. Provides cryptographically strong pseudo-random hypervectors with three concrete properties absent from conventional pseudo-random vector generators used in HDC literature (Mersenne Twister, xorshift, etc.):

1. **Cross-implementation determinism.** BLAKE3 has a fully-specified byte-level reference; every conformant implementation produces identical output for identical seed and length. Mersenne Twister, by contrast, is sensitive to seeding conventions and warm-up routines that vary by library.
2. **Replay-from-key.** A hypervector can be regenerated from its semantic key without storing the vector itself. This enables an implementation to discard cached encodings under memory pressure and re-derive them on demand; the eviction-and-replay mechanism is contemplated as a future optimisation of the present invention and is not part of the SHA 17334f8 embodiment, but the deterministic-derivation property that *enables* it is fully present.
3. **Cryptographic distribution quality.** BLAKE3 output is indistinguish-

able from uniform random under standard cryptographic assumptions. This guarantees that the expected similarity of two independently-seeded random hypervectors is exactly 0.5 with the predicted variance, removing a class of correlation artefacts that affect cheaper PRNGs.

6.5 Encoders (100)

Three encoders convert non-vector inputs into hypervectors. Each encoder calls only the primitives of §6.3 and §6.4, so its output is also bit-exact across implementations.

6.5.1 String Encoding (110) The string encoder maps a UTF-8 string to a hypervector via:

```
function encodeString(s: string, D: integer) -> hypervector:
    let domain_prefix = utf8_bytes("str:")
    let input          = domain_prefix || utf8_bytes(s)
    let seed           = BLAKE3(input)
    return randomHv(seed, D)
```

The domain prefix "str:" separates the string-encoder namespace from the level-encoder namespace (§6.5.2) and the role-namespaces conventions used in higher-level encoders. Two strings that differ in even a single byte produce hypervectors that are approximately orthogonal (expected similarity 0.5), giving exact-match key semantics without collision.

Technical Effect. Exact-match content-addressable encoding with zero false-positive collision rate (under BLAKE3 cryptographic assumptions) and zero storage overhead for the encoding state — no vocabulary, no learned parameters, no tokeniser. This is in contrast to embedding-based exact-match systems in which two literally identical strings can produce different float-32 vectors due to non-deterministic CUDA kernel scheduling, and where the encoding function itself occupies hundreds of megabytes (a learned tokeniser + an embedding matrix).

6.5.2 Embedding Encoding (Thermometer + Random Projection) (120)

The embedding encoder maps a real-valued vector (e.g., the float-32 output of a

foundation-model encoder) into a binary hypervector while approximately preserving cosine similarity as Hamming similarity. The thermometer-quantisation and random-projection approach to mapping real-valued vectors to binary hyperdimensional vectors has academic precedent in Imani et al. (2017, “Voicehd: Hyperdimensional computing for efficient speech recognition”, ICRC 2017) and Rahimi et al. (2016, “Hyperdimensional biosignal processing”, BioCAS 2016) for biosignal processing. The contribution of the present invention over these is the **specific deterministic form** of the per-(coordinate, level) hypervector derivation: the binding of the ASCII-encoded “lv1:i:level” seed to BLAKE3 followed by BLAKE3-XOF, which renders the encoder byte-identical across heterogeneous language runtimes — a property that no prior thermometer-random-projection encoder claims, because prior systems used non-cryptographic pseudo-random generators whose state diverges across language libraries.

The encoder is further distinguished from the *sign-LSH* / *SimHash* family of binary-fingerprint constructions (Charikar 2002, “Similarity Estimation Techniques from Rounding Algorithms”, STOC) and from the *sparse-sign random-projection* construction of Achlioptas (2003, “Database-friendly Random Projections: Johnson-Lindenstrauss with Binary Coins”, *J. Comput. Syst. Sci.*) on three independent points. First, sign-LSH and Achlioptas-style sparse projection produce **one bit per projection** applied to the *raw* real-valued vector, with no intermediate quantisation; the present encoder produces a full D-bit hypervector per (coordinate, level) pair and accumulates by XOR over N coordinates, so the algebraic surface is fundamentally different (binary-thermometer-then-bind, rather than binary-projection-then-concatenate). Second, neither Charikar 2002 nor Achlioptas 2003 specifies a cryptographic extendable-output function or a published wire format pinning the projection matrix across implementations; the seed conventions in those works are left to the implementor, which is precisely the source of cross-runtime divergence that the present invention eliminates. Third, the bit-budget per input scalar in sign-LSH is 1, whereas the present encoder’s bit budget per input scalar is D (with the redundancy absorbed by majority-bundling at the substrate level); this difference materially changes the recall-quality tradeoff at small D and is one of the reasons the present encoder achieves bit-exact agreement with cosine similarity over the [0.5, 1.0] range (per the Phase-0 validation notebook) where sign-LSH gives only

a probabilistic Hamming-vs-cosine correspondence.

The algorithm is a binary thermometer encoding with random projection per coordinate:

```
LEVELS = 100
```

```
function encodeEmbedding(embedding: array of float, D: integer) -> hypervector
  let acc = byte array of length D, all zero          # additive identity
  for i in 0..length(embedding)-1:
    let raw      = embedding[i]
    let clamped  = max(-1.0, min(1.0, raw))           # clamp to [-1, 1]
    let level    = round((clamped + 1.0) * (LEVELS - 1) / 2.0) # integer
    let seed_in  = utf8_bytes("lvl:" || str(i) || ":" || str(level))
    let seed     = BLAKE3(seed_in)
    let hv_lvl   = randomHv(seed, D)
    acc := bind(acc, hv_lvl)                          # XOR into accumulator
  return acc
```

Operational explanation. Each coordinate i of the input embedding is first clamped into $[-1, 1]$ to bound the level index; clamping is the chosen behaviour for out-of-range values rather than rejection, because foundation-model encoders occasionally produce values slightly outside the nominal range. The clamped value is then quantised to one of $L = 100$ levels by the affine map $\text{level} = \text{round}((\text{clamped} + 1) * (L - 1) / 2)$, which sends $-1.0 \rightarrow 0$, $0.0 \rightarrow 50$ (approximately, depending on rounding), and $+1.0 \rightarrow 99$. For each (coordinate-index i , quantised-level level) pair, a unique deterministic hypervector is derived by hashing the ASCII string "lvl:i:level" with BLAKE3 and expanding via `randomHv`. All such per-coordinate hypervectors are XOR-accumulated into the final output. The XOR accumulation is order-invariant (XOR is commutative and associative), so the encoder produces identical output for any permutation of the *coordinate index order* — which is correct, because the embedding’s coordinate indices are themselves part of the seed and therefore preserved.

Worked example. Let $D = 8$ and `embedding = [0.3, -0.7, 0.0, 0.9]`. The

quantised levels are: - $i = 0$: `clamped = 0.3`, `level = round((1.3 * 99) / 2)`
`= round(64.35) = 64` - $i = 1$: `clamped = -0.7`, `level = round((0.3 * 99) /`
`2) = round(14.85) = 15` - $i = 2$: `clamped = 0.0`, `level = round((1.0 * 99)`
`/ 2) = round(49.5) = 50` - $i = 3$: `clamped = 0.9`, `level = round((1.9 * 99)`
`/ 2) = round(94.05) = 94`

The encoder then computes:

```

hv_0 = randomHv(BLAKE3("lvl:0:64"), 8)
hv_1 = randomHv(BLAKE3("lvl:1:15"), 8)
hv_2 = randomHv(BLAKE3("lvl:2:50"), 8)
hv_3 = randomHv(BLAKE3("lvl:3:94"), 8)
output = hv_0 XOR hv_1 XOR hv_2 XOR hv_3

```

A perturbation of the input — say, changing 0.3 to 0.31 — would either leave level 0 at 64 (no change to encoding) or shift it to 65 (one summand changes, output changes in approximately half of the bits where `hv[0:64]` and `hv[0:65]` differ). Because adjacent levels yield independent random hypervectors, the encoding does *not* preserve continuity within a single coordinate; rather, it preserves *similarity* in the aggregate: two embeddings that are close in the input space share most of their level seeds and therefore most of their XOR summands, resulting in low Hamming distance.

(6.5.2.1) Note on rounding. Cross-implementation identity requires that `round((x + 1.0) * (L - 1) / 2.0)` produce the same integer in every language for the same `x`. Because `x` is clamped to `[-1, 1]`, the value to be rounded lies always in `[0, L - 1]` and is therefore **non-negative**, so any rounding rule that resolves half-integers consistently for non-negative inputs is admissible. The reference TypeScript binding (`packages/core-ts/src/encode.ts`) uses `Math.round`, which is IEEE 754 round-half-up (round-half-toward-positive-infinity). The reference Rust binding (`packages/core-rs/src/encode.rs`) uses `f32::round`, which is round-half-away-from-zero. These two rules **agree for all non-negative inputs** and are therefore interoperable in the present encoder. Implementations targeting IEEE 754 binary32 inputs **MUST** additionally normalise the order of arithmetic operations to the canonical bracketing `((clamped + 1.0) * (L - 1)) / 2.0` — that is, the multiplication is performed before the division — to prevent

associativity-related divergence; this canonical bracketing is used by both reference bindings and is recited in SPEC.md §A.1, together with the outer `clamp(level, 0, L - 1)` that absorbs the half-integer rounding-tail case where the three rules (round-half-up, round-half-away-from-zero, round-half-to-even) could nominally differ. The conformance corpus of §6.10 verifies byte-identical encoder output across both reference bindings.

Technical Effect. Maps `f32` embeddings into binary hypervectors with two measurable advantages over storing the raw `f32` embeddings:

1. **Storage reduction.** A 1024-dimensional `f32` embedding occupies 4096 bytes; encoding it into a $D = 10000$ binary hypervector occupies 1250 bytes — a $3.27\times$ reduction. For higher-dimensional source embeddings (e.g., the 4096-dimensional embeddings emitted by certain foundation-model encoders), the reduction approaches $13\times$ before considering any further compression.
2. **Similarity preservation.** Empirically, two embeddings with cosine similarity s produce hypervectors with Hamming similarity approximately s for s in the range $[0.5, 1.0]$, the range that matters for retrieval; the correspondence is documented in the Phase-0 validation notebook (`notebooks/phase0_hdc_validation.ipynb`). This means existing nearest-neighbour pipelines can be migrated to the binary substrate without re-training any upstream encoder.
3. **Per-query throughput.** Once the encoded substrate is in place, similarity computation runs at approximately $D / 64 = 156$ 64-bit-word operations per pair at $D = 10000$ (on a binding using the contemplated packed-bit form of §6.2.4), versus approximately 1024 floating-point multiply-accumulates plus two `sqrt` calls per pair for a 1024-dimensional float-32 cosine — an order-of-magnitude reduction in arithmetic operations and a comparable reduction in energy cost per query on commodity CPUs.

This is materially superior to Product Quantisation (FAISS-PQ), which requires a training step over a representative corpus, suffers from training-corpus distribution shift, and is not deterministic across hardware (FAISS-PQ uses BLAS kernels that are not reproducible across CPU SIMD widths).

6.5.3 Text Encoding (Permutation-Positional N-grams) (130, 140) For natural-language text, three encoders are provided. Permutation-positional n-gram encoding for text is documented in the academic HDC literature (Najafabadi et al. 2016, “HDC for text classification”). The present invention’s contribution over these is the combination of (i) the deterministic `encodeString` per-word hypervector of §6.5.1, (ii) the normative bit-rotation permutation of §6.3.3, and (iii) the deterministic-tiebreaker bundle of §6.3.1, all of which together render the entire text-encoder output byte-identical across implementations. Prior HDC text encoders make no such cross-platform-reproducibility claim.

(6.5.3.1) Bag-of-words encoder (130).

```
function encodeBagOfWords(text, D, min_word_length = 3) -> hypervector:
  let words = tokenise_lowercase(text, min_word_length)
  if length(words) == 0:
    return encodeString(text, D) # degenerate fallback
  return bundle([ encodeString("word:" || w, D) for w in words ])
```

Tokenisation `tokenise_lowercase` lowercases the input, splits on the regular expression `[^a-z0-9]+`, and discards tokens of length less than `min_word_length` (default 3; the rule discards single-letter tokens such as “a” and two-letter tokens such as “is”, “of”, “to”, “in”).

(6.5.3.2) Word-level n-gram encoder (140). This is the permutation-positional encoder. Each n-gram is encoded by binding the per-word hypervectors with permutation distances equal to their positions within the n-gram, then the n-grams are bundled:

```
function encodeWordNgrams(text, D, n) -> hypervector:
  require n >= 1
  let words = tokenise_lowercase(text)
  if length(words) < n:
    return encodeBagOfWords(text, D) # fallback
  if n == 1:
    return encodeBagOfWords(text, D)
  let ngrams = []
  for i in 0..length(words) - n:
```

```

let acc = encodeString("word:" || words[i], D)
for j in 1..n-1:
    let term = permute(encodeString("word:" || words[i + j], D), j)
    acc = bind(acc, term)                # XOR
ngrams.append(acc)
return bundle(ngrams)

```

Worked example: “alpha beta” versus “beta alpha” with $n = 2$.

- For “alpha beta”: one n-gram, computed as `bind(encodeString("word:alpha"), permute(encodeString("word:beta"), 1))`.
- For “beta alpha”: one n-gram, computed as `bind(encodeString("word:beta"), permute(encodeString("word:alpha"), 1))`.

Because `permute(x, 1)` is approximately orthogonal to `x`, the two outputs are approximately orthogonal to each other — Hamming similarity approximately 0.5. In contrast, a bag-of-words encoding of the same two strings would produce *identical* hypervectors, because XOR-bundle is permutation-invariant in its inputs. The permutation-positional encoder thus discriminates word order, while the bag-of-words encoder does not.

(6.5.3.3) Character n-gram encoder (140).

```

function encodeCharNgrams(text, D, n = 3) -> hypervector:
    let cleaned = lowercase(text)
    if length(cleaned) < n:
        return encodeString(text, D)
    let grams = []
    for i in 0..length(cleaned) - n:
        grams.append(encodeString("char:" || cleaned[i..i+n-1], D))
    return bundle(grams)

```

This tolerates typographical errors and matches partial substrings. As an arithmetic illustration, consider the query “elephnt” (7 characters; 5 trigrams: {"ele", "lep", "eph", "phn", "hnt"}) against the original “elephant” (8 characters; 6 trigrams: {"ele", "lep", "eph", "pha", "han", "ant"}). Three trigrams are common to both — {"ele", "lep", "eph"} — and the Jaccard overlap is $3 / (5 + 6 - 3) = 3 / 8 = 0.375$ of the combined trigram set. The corresponding character-n-gram-

encoded hypervectors therefore share most of the bundle summands closest to the query and produce a Hamming similarity materially greater than 0.5, even in the presence of a missing character.

Technical Effect. Provides order-sensitive, parameter-free text encoding with bit-exact cross-platform reproducibility. As compared with TF-IDF (requires a corpus-derived vocabulary), word2vec / GloVe (requires training), and transformer encoders (require GPU inference and produce non-deterministic float-32 outputs), the permutation-positional encoder runs in $O(\text{words} \times D)$ time per document, has no training step, has no vocabulary, has no learned parameters, and produces byte-identical output in every supported language runtime. The character n-gram variant additionally provides typo-tolerance without any spell-checker dependency.

6.6 Store: Item Management and Recall (300)

The **store** (300) is the user-facing public API. It exposes content-addressable storage in which the addressing space is the hypervector space $\{0, 1\}^D$. Item retrieval is mediated by the cleanup-memory index (310) and co-activation tracking is performed by the tracker (320).

(6.6.1) The Item data structure. Each stored item carries:

Field	Type	Notes
id	UUIDv7 string	Auto-generated on put unless supplied; UUIDv7 sorts lexicographically by creation time
key	hypervector of dimension D	The address in associative space
value	opaque byte string, ≤ 16 MiB	Configurable cap up to 256 MiB; larger payloads belong in a blob store
tags	list of UTF-8 strings	Arbitrary labels; used by recall filters
metadata	map of string $\rightarrow$ scalar	Scalar = string, integer, float, boolean, or null
createdAt	unsigned 64-bit Unix milliseconds	Preserved across put upsert

Field	Type	Notes
<code>lastAccessedAt</code>	Assigned 64-bit Unix milliseconds	Updated on every recall hit
<code>accessCount</code>	unsigned 32-bit integer	Incremented on every recall hit
<code>cold</code>	boolean	Set by <code>consolidate</code> when the item meets cold-eligibility criteria; boolean is preferred over a multi-level coldness score because the contemplated alternate-attic relocation step (v0.2.0) is a discrete operation, not a gradient one

The logical store-of-items is exposed as `store.items`, a map keyed by `id` and providing at minimum the operations `get(id) -> Item | null`, `set(id, item) -> void`, `delete(id) -> boolean`, and iteration over its entries.

(6.6.2) Public API.

<code>open(config)</code>	-> Store
<code>put(store, key, value, opts?)</code>	-> Item
<code>recall(store, cue, opts?)</code>	-> list of Match
<code>get(store, id)</code>	-> Item
<code>delete(store, id)</code>	-> boolean
<code>consolidate(store, opts?)</code>	-> ConsolidationReport
<code>snapshot(store)</code>	-> bytes (KMF)
<code>restore(bytes)</code>	-> Store

In all cases, `key` (in `put`) and `cue` (in `recall`) may be supplied as either (i) a UTF-8 string, in which case `encodeString` is applied internally, (ii) a real-valued array, in which case `encodeEmbedding` is applied, or (iii) a hypervector, which is used directly.

(6.6.3) The `put` operation. The store exposes a standard upsert-on-put operation that preserves `createdAt` and `accessCount` on existing identifiers and assigns a new UUIDv7 identifier when none is supplied. Concretely:

```
function put(store, key, value, opts):
```

```

let bytes = (value as bytes)
require length(bytes) <= store.valueCapBytes
let hv    = toHypervector(key, store.dimension)
let now   = current_unix_millis()
let id    = (opts.id provided) ? opts.id : uuidv7(now)
let existing = (opts.id provided) ? store.items.get(id) : null
let item = Item {
  id, key = hv, value = bytes,
  tags    = opts.tags or [],
  metadata = opts.metadata or {},
  createdAt = (existing != null) ? existing.createdAt : now,
  accessCount = (existing != null) ? existing.accessCount : 0,
  lastAccessedAt = now,
  cold = false,
}
store.items.set(id, item)
return item

```

If the supplied `id` already exists in the store, the operation is **upsert** with `createdAt` preserved.

(6.6.4) The **recall** operation (cleanup memory, 310).

```

function recall(store, cue, opts):
  let topK    = opts.topK    or store.defaultTopK          # default 10
  let minSim  = opts.minSimilarity or store.defaultMinSim  # default 0.5
  let cueHv   = toHypervector(cue, store.dimension)
  let candidates = []
  for each item in store.items:
    if opts.filter is provided and not opts.filter(item):
      continue
    candidates.append({ id: item.id, key: item.key })
  let hits    = cleanupSearch(candidates, cueHv, topK, minSim)
  let now     = current_unix_millis()
  let matches = []
  for each hit in hits:

```

```

    let item = store.items.get(hit.id)
    item.accessCount      += 1
    item.lastAccessedAt   = now
    item.cold              = false
    matches.append({ item, similarity: hit.similarity })
if length(matches) >= 2:
    store.tracker.record([m.item.id for m in matches])      # co-activation
return matches

```

The `cleanupSearch` routine is brute-force linear scan in the reference Phase-1 implementation:

```

function cleanupSearch(entries, cue, topK, minSimilarity):
    let out = []
    for each entry in entries:
        require length(entry.key) == length(cue)
        let s = similarity(entry.key, cue)
        if s >= minSimilarity:
            out.append({ id: entry.id, similarity: s })
    sort out by (similarity descending, id lexicographic ascending)
    return out[0..topK-1]

```

Tiebreaking on equal similarity is by lexicographic ascending id, which is identical across implementations because UUIDv7 yields the same ASCII representation in every language.

(6.6.5) Optional locality-sensitive hashing layer (contemplated, Phase 2 / v0.2.0). The brute-force scan of §6.6.4 is the normative Phase-1 implementation and is the embodiment present at SHA 17334f8. SPEC.md §4.2 expressly demarcates the Phase 1 brute-force scan as normative for v0.1.0 and demarcates an LSH-or-learned-index hybrid as contemplated and non-normative for a future v0.2.0 spec revision. The following paragraph describes a planned Phase 2 enhancement that is **not part of the present embodiment** but is disclosed for purposes of describing the inventor’s contemplated mode of implementation. The Phase 1 brute-force scan is acceptable up to approximately 10^5 items at $D = 10000$ on a commodity CPU. Phase 2 introduces a Locality-Sensitive Hashing (“LSH”) layer

for higher item counts, structured as a band-of-tables of random-bit-position projections (the standard Hamming-space LSH construction of Indyk and Motwani 1998, refined by Andoni and Indyk 2008), returning a candidate set that is then ranked by the same brute-force scan over the reduced set. The Phase 2 LSH layer is required to produce **the same top-k** as the brute-force scan for any `(cue, k, store state)` triple; LSH is permitted only to widen the candidate set, never to alter the ranking. This invariant is what makes the cross-binding conformance corpus possible and is what distinguishes the contemplated layer from a Hierarchical-Navigable-Small-World (“HNSW”) index (Malkov and Yashunin 2020, *IEEE TPAMI*), which does not guarantee identical top-k across implementations because its graph-construction order is non-deterministic. The LSH layer is disclosed as the inventor’s contemplated mode for the Phase 2 cleanup memory; it is not the subject of any present claim and may be claimed in a subsequent divisional application.

Technical Effect. Provides content-addressable storage with three measurable improvements over conventional key-value stores plus a separate vector index (the architecture used by, e.g., Redis + FAISS sidecar, or PostgreSQL + pgvector):

1. **Single index, single substrate.** The hypervector *is* the address; there is no separate vector index to keep coherent with the item map. This eliminates a class of consistency bugs in which the vector index falls out of sync with the canonical record store.
2. **Filter-then-rank semantics.** The optional `filter` predicate is applied before similarity ranking, which means filter conditions over metadata never produce empty result sets due to top-k truncation — a known failure mode of vector databases that apply filter conditions *after* nearest-neighbour ranking (publicly documented for `pgvector` in its issue tracker and for Pinecone in its community forum; the failure mode manifests when the top-k set returned by the ANN index contains no items satisfying the metadata filter, even though such items exist further down the similarity ordering).
3. **Deterministic recall under ties.** Identical recall results across replicas, even when multiple items share the top similarity score, because the tiebreaker is lexicographic on `id` and `ids` are UUIDv7 ASCII strings identical across runtimes.

6.7 Consolidation (Replayable Hebbian) (330)

The consolidation procedure (330) periodically reshapes the substrate so that items frequently recalled *together* become *more similar in Hamming space*. This is a Hebbian principle (“neurons that fire together, wire together”) adapted to the binary hypervector substrate, with the critical innovation that the procedure is **fully deterministic and replayable**.

The Hebbian principle (Hebb 1949, *The Organization of Behavior*) and its application to associative memory (Hopfield 1982, “Neural networks and physical systems with emergent collective computational abilities”; Kanerva 1988, *Sparse Distributed Memory*) are prior-art foundations. The present invention’s novel contribution is the **deterministic, salt-driven, BLAKE3-ordered, alternate-direction bit-flip formulation** of §6.7.2, which renders the Hebbian update fully replayable from a snapshot + access-log pair. No prior Hebbian consolidation procedure — including the modern continuous-Hopfield work of Ramsauer et al. (2020, “Hopfield Networks Is All You Need”) — supports byte-identical cross-platform replay, because all such prior procedures operate on floating-point weights whose update order and rounding depend on hardware-specific BLAS or auto-differentiation kernels.

6.7.1 Co-activation Tracking (320) A sliding window of size W (default $W = 1000$) records the IDs of items returned together by each `recall` call. For each pair (a, b) of items co-occurring in the window, a counter $c(a, b)$ is maintained:

```
class CoactivationTracker:
    window          : list of (list of item-ids)
    pair_counts     : map of (id_a, id_b) -> integer
    size            : integer (window capacity)

    function record(ids):
        if length(ids) < 2:
            window.append(ids)
            trim()
            return
        let sorted_ids = sort_ascending(ids)           # canonical order: lex
        for i in 0..length(sorted_ids)-1:
```

```

        for j in i+1..length(sorted_ids)-1:
            pair_counts[(sorted_ids[i], sorted_ids[j])] += 1
    window.append(sorted_ids)
    trim()

function trim():
    while length(window) > size:
        let expired = window.pop_front()
        if length(expired) < 2: continue
        for i in 0..length(expired)-1:
            for j in i+1..length(expired)-1:
                decrement pair_counts[(expired[i], expired[j])]
                if the count reaches 0: remove the entry

function pairsAtOrAbove(threshold):
    let out = []
    for each ((a, b), count) in pair_counts:
        if count >= threshold:
            out.append({ a, b, count })
    sort out by (count descending, a ascending, b ascending)
    return out

```

The sorted-pair canonicalisation `sort_ascending(ids)` ensures (a, b) and (b, a) are recorded as the same pair, and gives deterministic ordering when multiple pairs share the same count.

6.7.2 Binding Pull (the Hebbian step) — Fig. 4 steps 401–407 When a pair (a, b) has co-activation count $\geq$ `pullThreshold` (default 32), the keys of items a and b are pulled closer in Hamming space by flipping a bounded number of disagreeing bits. Crucially, the *choice of which bits to flip* is deterministically derived from a salt that increments per consolidation pass:

```

function pullCloser(a: hypervector, b: hypervector, maxSimDelta: float, salt: int):
    -> { newA, newB, bitsFlipped }:
    require length(a) == length(b)

```

```

let D = length(a)

# 1. Find disagreeing bit positions.
let disagree = []
for i in 0..D-1:
    if a[i] != b[i]:
        disagree.append(i)

# 2. Bound the number of flips: at most floor(maxSimDelta * D), but at least 1
let maxFlips = max(1, floor(maxSimDelta * D))      # default maxSimDelta = 0.5
let toFlip    = min(length(disagree), maxFlips)

# 3. Derive a deterministic ordering over the disagreeing bits via BLAKE3.
let seedBytes = u64_le(salt)                      # 8 bytes
let digestLen = max(64, toFlip * 4)               # see note below
let digest     = BLAKE3-XOF(seedBytes, digestLen)

# Sort the `disagree` list of bit indices `b` by the composite key
# (digest[b mod length(digest)], b) in ascending order. The first
# component provides the BLAKE3-driven pseudo-random ordering; the
# second is the lexicographic tiebreaker when two indices hash to the
# same digest byte.
sort disagree by key b -> (digest[b mod length(digest)], b) ascending

# 4. Apply alternate-direction flips: even index toward b, odd index toward a
let aOut = copy(a)
let bOut = copy(b)
for k in 0..toFlip-1:
    let bit = disagree[k]
    if k mod 2 == 0:
        aOut[bit] := b[bit]                      # a moves toward b
    else:
        bOut[bit] := a[bit]                      # b moves toward a

```

```
return { newA: aOut, newB: bOut, bitsFlipped: toFlip }
```

The `maxSimDelta` parameter (default 0.02) bounds the per-pass similarity drift: at $D = 10000$, the operation flips at most $\text{floor}(0.02 * 10000) = 200$ bits, which moves `sim(a, b)` by at most 0.02. This bound prevents a single consolidation pass from collapsing two distinct items into the same address; consolidation acts as a slow drift rather than an abrupt merge.

The `digestLen = max(64, toFlip * 4)` rule is chosen so that (i) the factor of 4 provides at least one digest byte per disagreeing bit position participating in the flip (preventing the modulo wrap-around from concentrating the sort key onto a small alphabet and creating excessive collisions in the BLAKE3-driven ordering), and (ii) the `max(64, ...)` floor ensures BLAKE3-XOF is exercised with a non-trivial output length even when `toFlip` is small.

The **alternate-direction** flip rule ensures that both `a` and `b` move approximately equally — neither one is dragged entirely toward the other. The choice of which disagreeing bits to flip first is determined by `BLAKE3-XOF(u64_le(salt))`, where `salt` is the per-pass generation counter maintained by the store. Two replicas running on the same access log with the same initial state and the same salt sequence flip *exactly* the same bits in *exactly* the same order.

6.7.3 Cold-Item Flagging Items satisfying both `lastAccessedAt < (now - coldDays * 86_400_000)` and `accessCount < coldMinAccess` are flagged as cold:

```
function flagColdItems(items, config, now):
    let cutoff = now - config.coldDays * 86_400_000      # millis
    return [ it.id for it in items
        if it.accessCount < config.coldMinAccess
        and it.lastAccessedAt < cutoff ]
```

Cold-flagged items remain in the store (the Phase-1 behaviour present at SHA 17334f8); subsequent versions of the invention may relocate cold items to an “attic” sub-store from which they can be cheaply re-instantiated on a near-hit. This attic relocation, including its bundled-summary representation and its associated KMF `attic_block` (445) per §6.8.7 below, is **reserved for v0.2.0 of the spec** and is

not part of the present embodiment.

6.7.4 Replay Determinism The consolidation procedure satisfies the following replay invariant:

Given (i) the same initial substrate state S_0 , (ii) the same sequence of **put** and **recall** operations, and (iii) the same per-pass salt sequence, the consolidated state S_n is **byte-identical** across all conformant implementations.

The reasons are: - Co-activation tracking uses sorted IDs and integer counters, both of which are platform-independent. - The pair-selection ordering for the pull step uses (**count desc**, **a asc**, **b asc**), all integer / lexicographic comparisons. - The bit-flip ordering is `BLAKE3-XOF(u64_le(salt))`, which is byte-identical across implementations. - The flip rule (**even** $\rightarrow$ **a** := **b**[**bit**], **odd** $\rightarrow$ **b** := **a**[**bit**]) is integer arithmetic with no floating-point.

Each consolidation pass writes its salt value into the persisted access log as a u64 little-endian integer at a fixed offset within the pass record, so that a verifier replaying the log can re-derive the bit-flip ordering deterministically. A **restore module** reads a KMF snapshot together with the append-only access log (the log records, for each operation, a typed entry of one of {**put**, **recall**, **consolidate**} with full byte-level parameters including the per-pass consolidation salt), and re-executes each entry in order against the loaded snapshot to reconstruct the current state byte-for-byte. The access-log persistence and restore-module facility is contemplated at SHA 17334f8; its full apparatus form is the subject of dependent claim 15 of independent claim 9.

Technical Effect. Provides deterministic, replayable index reshaping — a system-level property that no conventional vector database supports. Specifically:

1. **Snapshot + access-log replay.** Given a KMF snapshot of state S_0 and a log of subsequent operations, an auditor can re-run the consolidation procedure and arrive at the *byte-identical* current state. This enables verifiable audit of memory evolution — useful in regulated environments (medical agent memory, legal discovery) where the provenance of a stored item must be reconstructable.
2. **Multi-replica consistency without consensus.** Two replicas processing

the same operation log converge to byte-identical state without any consensus protocol between them. This eliminates the need for Paxos / Raft style co-ordination for the consolidation pass, an order-of-magnitude reduction in operational complexity compared with conventional replicated databases.

3. **Deterministic Hebbian compaction.** Conventional neural Hebbian updates use floating-point weight changes which are not bit-exact across CPU architectures, GPU drivers, or BLAS implementations. The binary flip-based formulation here is exact integer arithmetic plus BLAKE3, neither of which has hardware-dependent variants.

6.8 KMF Wire Format (Smritidb Memory Format) (400)

The Smritidb Memory Format (“KMF”, element 400) is the persistent, implementation-independent wire format for a Smritidb substrate. The format is positioned for the same standards trajectory as Apache Parquet (columnar tabular storage; Apache Parquet Format 2.6) and Apache Iceberg (table-level transactional metadata): a canonical byte-layout that any conformant implementation can read and write. The use of a fixed header magic (410), a u64 header offset (412), a trailing footer (470), and per-block hashes (450) is structurally similar to the Parquet footer layout and the Iceberg table-metadata layout. The novelty of KMF lies in **the binding of this generic container structure to the binary hypervector substrate**: specifically, the MSB-first column-major `hv_block` (420) of §6.8.2 and the deterministic per-block BLAKE3 digest at the substrate-bit level, which together permit cross-implementation byte-identical reads of a hyperdimensional substrate.

6.8.1 File Layout A KMF file is structured as five contiguous sections:

```
+-----+ offset 0
| (a) Magic bytes (410): "KMF\0" (0x4B 0x4D 0x46 0x00) - 4 bytes |
+-----+ offset 4
| (b) Spec version (411): "0.1.0\0" (ASCII, NUL-pad) - 6 bytes |
+-----+ offset 10
| (c) Header offset (412): u64 little-endian - 8 bytes |
```

+-----+ offset 18		
(d) Data blocks	(variable length; see §6.8.2 - §6.8.4)	
- hv_block	(420)	
- meta_block	(430)	
- value_block	(440)	
- attic_block	(445) [contemplated, v0.2.0; see §6.8.7]	
+-----+ offset (h		
(e) Header	(460) (JSON-encoded; zstd compression contemplated)	
{ spec_version, dimension, item_count, created_at,		
index: [{ kind, offset, length, blake3 (450) }, ...] }		
+-----+ offset (f		
(f) Trailer magic	(470): "FMK\0" (0x46 0x4D 0x4B 0x00) - 4 bytes	
+-----+ offset (f		

The header offset stored at byte 10 is a u64 little-endian integer that locates the start of the header. To read a KMF file, an implementation **MUST**:

1. Verify bytes 0–3 equal "KMF\0".
2. Verify the last four bytes of the file equal "FMK\0" — this trailer guard detects truncation **before** any offset is dereferenced.
3. Read the spec version at bytes 4–9 and reject if the major version exceeds the supported major.
4. Read the u64 header offset at bytes 10–17.
5. Read the header from `header_offset` up to `(file_length - 4)`, JSON-decode it.
6. For each block referenced in the header's `index` field, slice the bytes [`offset`, `offset + length`) from the data region, compute BLAKE3 over those bytes, and verify the result equals the `blake3` field stored in the index. Any mismatch raises a `CorruptSnapshot` error.
7. Only after all block hashes verify does the reader deserialise the block contents.

6.8.2 The hv_block (420) The `hv_block` (420) stores the hypervector keys in column-major MSB-first packed-bit form. For a substrate of `n` items at dimension `D`, the block has length `n × ceil(D / 8)` bytes. Item `k`'s hypervector occupies bytes `k × ceil(D / 8)` to `(k + 1) × ceil(D / 8) - 1`. Within each item's region, bit `i`

is at byte $\text{floor}(i / 8)$, bit offset $7 - (i \bmod 8)$ (MSB-first), identical to the in-memory packed representation. When the substrate contains a single-item snapshot (for instance, a singleton bundle output emitted directly as a one-row KMF file), the `hv_block` consists of that one hypervector's $\text{ceil}(D / 8)$ bytes, and the per-block BLAKE3 (450) in the header index of §6.8.5 is the BLAKE3 of those bytes, providing the integrity-hash emission contemplated by claim 5.

Worked byte-level example: $D = 16$, three items. Each hypervector occupies $\text{ceil}(16 / 8) = 2$ bytes. The block occupies $3 \times 2 = 6$ bytes. Suppose the three hypervectors are:

```

item 0: bits = 1 0 1 1 0 0 1 0 | 1 1 0 0 0 0 0 1   -> bytes 0xB2 0xC1
item 1: bits = 0 0 0 0 0 0 0 0 | 1 1 1 1 1 1 1 1   -> bytes 0x00 0xFF
item 2: bits = 1 1 1 1 1 1 1 1 | 0 0 0 0 0 0 0 0   -> bytes 0xFF 0x00

```

The `hv_block` then comprises the bytes:

```
B2 C1 00 FF FF 00
```

The very same byte sequence is what is written by the Rust binding, the TypeScript binding, the Python binding, the Kotlin / Swift bindings via UniFFI, and the WebAssembly build of the Rust binding. The byte-exact identity is verifiable by computing BLAKE3 over the six bytes and comparing against the header's stored hash.

6.8.3 The `meta_block` (430) The `meta_block` (430) stores per-item metadata as a **JSON-encoded array** — this is the normative Phase-1 encoding and is the encoding used by the reference implementation at SHA 17334f8 (see `packages/core-ts/src/kmf.ts`, which calls `JSON.stringify` on the metadata array). A Phase-2 variant employing length-prefixed MessagePack records is **contemplated** for streaming efficiency and is **not part of the present embodiment**. Fig. 2 of the drawings annotates this block as “JSON (Phase 1; MessagePack contemplated for Phase 2)”, in agreement with the present specification. Each record contains:

```

{
  "id":          "<uuid-v7-string>",
  "tags":        [<string>, ...],

```

```

"metadata":      { <key: string> -> <scalar> },
"createdAt":     <unix-millis>,
"accessCount":   <u32>,
"lastAccessedAt": <unix-millis>
}

```

Records appear in the same order as the hypervectors in the `hv_block`, so item `k`'s metadata is the `k`-th entry.

6.8.4 The `value_block` (440) The `value_block` (440) stores opaque payloads in length-prefixed form. Each payload is preceded by a `u32` little-endian length, followed by exactly that many payload bytes. The values appear in item order.

```

+-----+-----+-----+-----+ ...
| u32 LE L0 | L0 bytes value | u32 LE L1 | L1 bytes value |
+-----+-----+-----+-----+ ...

```

The value block treats payloads as opaque bytes; the substrate does not interpret them. Higher layers (the `AgentMemory` example built atop the store, for instance) may impose JSON or any other encoding by convention, but the substrate guarantees nothing about value content.

6.8.5 Header (460) The header (460) is a JSON object. At SHA 17334f8 the reference implementation writes the header as a plain UTF-8 JSON byte sequence (see `packages/core-ts/src/kmf.ts`, where the header is produced by `encodeAscii(JSON.stringify(h))`). An optional Zstandard-compression layer over data-block bytes (analogous in apparatus form to compression of the header bytes themselves) is **contemplated** for a subsequent revision and is exposed by dependent claim 20 of independent claim 18 (which permits each data block to be compressed prior to BLAKE3 hashing); this compression layer is **not part of the SHA 17334f8 embodiment**. Its fields are:

Field	Type	Meaning
<code>spec_version</code>	string, exactly "0.1.0" in v0.1	The spec version this snapshot was written against
<code>dimension</code>	unsigned integer	The dimension <code>D</code>

Field	Type	Meaning
<code>item_count</code>	unsigned integer	Total number of items in the substrate
<code>created_at</code>	unsigned 64-bit Unix millis	Snapshot creation time
<code>index</code>	array of block references	One entry per data block (see below)

Each block reference is:

```
{
  "kind":    "hv_block" | "meta_block" | "value_block",
  "offset":  <byte offset into the file where the block starts>,
  "length":  <length of the block in bytes>,
  "blake3":  <hex-encoded 32-byte BLAKE3 digest of the block bytes>
}
```

The `blake3` field is the lower-case hex string of the 32 bytes returned by `BLAKE3(block_bytes)`. Implementations **MUST** verify each block against this digest before deserialising.

6.8.6 Trailer (470) The four-byte trailer (470) "FMK\0" (the reverse of the header magic) serves as an anti-truncation guard. Many file truncation incidents (interrupted writes, partial cloud-object downloads, abnormal process exit) produce files that begin correctly but are cut off mid-block. By requiring the trailer to be present *and at the end of the file*, the format guarantees that any truncation is detected before any offset in the file is trusted. The asymmetry between header magic "KMF\0" (410) and trailer magic "FMK\0" (470) additionally makes it impossible to confuse the file for a doubled or concatenated KMF stream.

6.8.7 The `attic_block` and the Compressed-Block Pipeline (contemplated) This subsection describes apparatus disclosed for purposes of best-mode contemplation that is **not part of the SHA 17334f8 embodiment** but supports the disclosure of dependent claim 22 of independent claim 18 (`attic_block`) and dependent claim 24 of independent claim 23 (Zstandard compressor coupled with BLAKE3 hasher).

(6.8.7.1) The `attic_block` (445). When the consolidation procedure of §6.7

flags an item as cold (see §6.7.3) and the contemplated v0.2.0 attic relocation step is enabled, the cold item’s hypervector is removed from the principal `hv_block` (420) and a *bundled summary* of one or more cold items is written into a dedicated `attic_block` (445). Each entry in the `attic_block` is a record of the form `{ source_item_ids: [<uuidv7-string>, ...], bundled_hypervector: <ceil(D/8) bytes> }`, where `source_item_ids` enumerates the item identifiers whose hypervectors were bundled (per §6.3.1) to produce the contained `bundled_hypervector`. The `attic_block` is included in the header (460) `index` array with `kind = "attic_block"`, an explicit byte offset, length, and BLAKE3 (450) digest computed over the attic block’s bytes — providing the same per-block integrity property as the principal `hv/meta/value` blocks. The attic mechanism enables a near-hit query against a cold item to be served by re-instantiating the item from its bundled summary, without retaining the full per-item hypervector in the principal substrate. This `attic_block` apparatus is the subject of dependent claim 22 of independent claim 18 (which provides for an optional attic block independently hashed and indexed); it is reserved for v0.2.0 of the spec.

(6.8.7.2) Zstandard-compressed block pipeline. When a block (any of `hv_block`, `meta_block`, `value_block`, or `attic_block`) is to be written in compressed form, the Zstandard compressor is coupled between the block-emit pipeline and the BLAKE3 hasher (450) such that the BLAKE3 digest recorded in the header (460) `index` is computed over the **compressed bytes** rather than the uncompressed ones. This pipeline ordering — “compress, then hash” — allows a reader to verify the integrity of the on-disk bytes before any decompression step, and accordingly to detect compression-stream corruption (e.g., a truncated zstd frame) before the decompressor is invoked. The reader correspondingly decompresses only after the per-block BLAKE3 verification has succeeded. This compress-then-hash pipeline ordering is the subject of dependent claim 24 of independent claim 23 (which provides for a Zstandard compressor coupled between a block-emit pipeline and the BLAKE3 hasher).

Technical Effect. The KMF format yields the following measurable improvements over conventional persistence approaches for vector databases (e.g., the proprietary on-disk formats of FAISS, Annoy, hnswlib, or the SQL-blob serialisations used by pgvector):

1. **Implementation-independent persistence.** A snapshot written by the Rust binding loads byte-for-byte into the TypeScript, Python, Kotlin, or Swift binding. This is empirically verified by the cross-binding conformance corpus (§6.10).
2. **Per-block cryptographic integrity.** Corruption (bit flips on disk, partial network reads, malicious tampering) is detected at block granularity via BLAKE3, before the corrupted block is interpreted. FAISS’s on-disk format, by contrast, has no integrity check; a single corrupted byte in an index file produces silently wrong nearest-neighbour results.
3. **Streaming-friendly layout.** The header sits at the *end* of the file, immediately before the trailer, but is located via the fixed-position `u64` header offset at byte 10. A reader can therefore (a) read the header without parsing any block, (b) decide which blocks to load (e.g., metadata only, or only the first *N* items’ hypervectors), and (c) seek directly to those blocks. This enables `O(1)` “what’s in this file?” queries without `O(file_size)` parsing — a property absent from formats that interleave metadata and payload (e.g., HDF5 with chunked datasets).
4. **Anti-truncation by construction.** The “FMK\0” trailer must be present at the file end, detectable by a single seek-to-end read, before any offset in the file is trusted. This eliminates a class of “partial write, garbage reads” bugs that plague append-only formats.

6.9 Persistence Adapters (500)

The KMF wire format defines *what* is written. Storage adapters define *where*. The invention provides a uniform `StorageAdapter` interface (500) — with methods `read_snapshot` (510), `write_snapshot` (520), `append_wal` (530), `read_wal` (540), `truncate_wal` (550), and `close` (560) — that any storage substrate may implement:

```
interface StorageAdapter:
```

```
    kind:    string                # informational
    read():  bytes or null         # the most recent snapshot, or null
    write(bytes): void            # atomically replace the snapshot
    remove?(): void               # delete the snapshot (optional)
```

The reference implementation provides four adapters:

(6.9.1) Memory adapter (610). An in-process, ephemeral adapter that holds the most recent snapshot bytes in an in-memory buffer. Used for testing and for transient sessions where persistence is not required.

(6.9.2) File-system adapter (620). Writes the snapshot atomically to a file on the local filesystem, using the standard `write-to-temp + rename` idiom that POSIX guarantees to be atomic within a single directory (on POSIX-conformant filesystems; on Microsoft Windows the equivalent guarantee is provided by `MoveFileEx` with the `MOVEFILE_REPLACE_EXISTING` flag). Crashes during write leave either the previous snapshot intact or the new snapshot complete; partial writes are never observable to a reader.

(6.9.3) SQLite adapter (630). Stores the snapshot bytes as a single BLOB row in a SQLite database, using SQLite’s Write-Ahead-Logging (“WAL”) journal mode (Hipp et al.; SQLite specification). Inherits SQLite’s ACID guarantees: a successful `write()` returns only after the snapshot is durable on disk; an interrupted process leaves the previous snapshot recoverable. Compatible with the Rust `rusqlite` bindings used by the Rust implementation, and with the WebAssembly SQLite build used by browser implementations. The novelty of the present invention over a bare SQLite-backed BLOB is **not the WAL mechanism itself** — which is a well-known SQLite feature in the prior art — but the **decoupling of the algorithmic substrate from the durable storage layer via the `StorageAdapter` interface (500)**, such that the same KMF byte stream can be redirected to any of the four reference adapters or to a user-supplied implementation without modification of the algorithmic core; the SQLite adapter merely composes the substrate over SQLite’s durability primitives, inheriting rather than re-implementing them.

(6.9.4) IndexedDB adapter (640). Stores the snapshot in a browser IndexedDB object store. Provides offline persistence for web applications without server round-trips. The adapter writes the entire snapshot as a single Blob value, which IndexedDB stores efficiently (typically referencing the bytes by pointer rather than copying).

A *higher-order* adapter helper `withPersistence(store, adapter, { debounceMs })` wraps a live store so that every `put`, `delete`, and `consolidate` call schedules an

auto-snapshot through the adapter, debounced by a configurable interval (default 100 ms). This coalesces burst writes into a single snapshot.

Technical Effect. Decouples the algorithmic substrate from the storage backend, enabling the same code to operate over heterogeneous storage substrates without modification:

1. **Backend substitution at runtime.** A library consumer chooses among the four reference adapters disclosed above (Memory (610), File-system (620), SQLite (630), IndexedDB (640)), or implements the `StorageAdapter` interface (500) against any further substrate (potential user-extension examples including but not limited to Amazon S3, Cloud Spanner, or Postgres-as-blob; these are not present implementations at SHA 17334f8), by passing a different `StorageAdapter` instance to the same store constructor. The core algorithm is unchanged.
2. **Inheritance of storage-substrate properties.** When the SQLite adapter is used, the substrate inherits ACID semantics for free; when the file-system adapter with atomic rename is used, the substrate inherits POSIX rename-atomicity; when the IndexedDB adapter is used, the substrate becomes available offline in any modern browser. This compositional reuse of platform-provided durability is materially superior to building bespoke durability into the substrate (the approach taken by purpose-built vector databases).
3. **Single wire format across backends.** Because every adapter operates on the same KMF byte stream, the substrate can be migrated between backends by exporting from one and importing into another, with no format-conversion step.

6.10 Cross-Implementation Bit-Exactness and Conformance Corpus (660)

The system specification requires that all conformant implementations agree at the byte level on the output of every primitive, every encoder, and every wire-format operation. To make this contract testable, the invention defines a **conformance corpus (660)** — a JSON file `tests/conformance/golden.json` that lists, for a fixed set of inputs, the expected outputs.

The technique of a JSON-encoded conformance corpus exercised by every implementation’s continuous-integration pipeline is borrowed from established standards-body practice — including the W3C Web Platform Tests and the IETF JSON-Patch conformance suite (RFC 6902 §11). The present invention’s contribution over these precedents is **the specific binding of such a corpus to the bit-level outputs of a hyperdimensional substrate**, including SHA-256-hashed packed-bit hypervectors as the canonical comparison artefact — a binding that no prior HDC system maintains.

(6.10.1) Corpus structure. The corpus is a JSON object containing typed test categories:

```
{
  "spec_version":  "0.1.0-draft",
  "generated_by":   "@tanvrit/smritidb (TS reference)",
  "random_hv":      [ { "seed_utf8", "dim", "sha256" }, ... ],
  "encode_string":  [ { "input", "dim", "sha256" }, ... ],
  "similarity_pairs": [ { "a_seed", "b_seed", "dim", "expected" }, ... ],
  "bind_round_trip": [ { "a_seed", "b_seed", "dim", "expected_similarity_to", ... ],
  "bundle":         [ { "seeds": [...], "dim", "sha256" }, ... ],
  "text_bag_of_words": [ { "text", "dim", "sha256" }, ... ],
  "text_char_ngrams": [ { "text", "dim", "n", "sha256" }, ... ]
}
```

Each `sha256` field is the lower-case hex SHA-256 hash of the raw output bytes of the corresponding hypervector. The reference TypeScript implementation generates the corpus by computing each output and writing the hash; every other binding *verifies* the corpus by recomputing the output and comparing its SHA-256 to the listed value. Use of SHA-256 (rather than BLAKE3) for the corpus hash is intentional: it cross-checks that the BLAKE3 used in `randomHv` has not been silently substituted with a different BLAKE3 variant, by introducing a second independent hash function whose libraries are unrelated to BLAKE3.

(6.10.2) Test categories at the SHA 17334f8.

Entry		
Category	count	Tested behaviour
<code>random_hv</code>	5	<code>randomHv(BLAKE3(seed_utf8), dim)</code> produces expected bytes
<code>encode_string</code>	3	<code>encodeString(input, dim)</code> produces expected bytes
<code>similarity_pairs</code>	3	<code>similarity(randomHv(a), randomHv(b))</code> lands within $1e-9$ of expected
<code>bind_round_trip</code>	1	<code>bind(bind(a, b), b) == a</code> (similarity exactly 1)
<code>bundle</code>	2	<code>bundle([randomHv(s) for s in seeds])</code> produces expected bytes
<code>text_bag_of_words</code>	2	<code>encodeBagOfWords(text, dim)</code> produces expected bytes
<code>text_char_ngrams</code>	2	<code>encodeCharNgrams(text, dim, n)</code> produces expected bytes

The corpus is intentionally small at SHA 17334f8 — namely **five random-hv vectors, three string-encodings, three similarity pairs, one bind round-trip, two bundles, two bag-of-words, and two char-ngrams, for eighteen total entries**. A passing corpus does not *prove* full correctness; it proves *agreement with the reference at these specific inputs*. The corpus expands with each spec revision.

(6.10.3) CI integration. Each binding’s continuous-integration pipeline runs the corpus as a mandatory test. A binding that fails any corpus entry is, by definition, non-conformant; it cannot be released under the Smritidb name. The Rust binding’s (720) conformance test lives at `packages/core-rs/tests/conformance.rs`; the Python binding’s (730) at `packages/smritidb-py/tests/test_smritidb.py`; the Kotlin (740) / Swift (750) binding’s at the UniFFI-generated test harness in `packages/smritidb-ffi/` (the UniFFI Interface Description Language (“UDL”) contract is at `packages/smritidb-ffi/src/smritidb.udl`); the TypeScript binding’s (710) at `packages/core-ts/src/conformance.test.ts` (where the reference itself dog-foods the corpus). The conformance-verification flow is summarised in **Fig. 5**, with the PASS outcome at (780) and the FAIL outcome at (790).

Technical Effect. Provides a verifiable, automated contract between independent language implementations:

1. **Drift prevention.** Cross-implementation drift is detected by any CI run, not by user-reported bugs in production. This eliminates a known and costly failure mode in conventional vector databases, where the C++ core and the Python bindings drift in floating-point handling across releases.
2. **Legal artefact for interoperability.** The corpus is a concrete, repository-resident, machine-verifiable file. It is the artefact that demonstrates — to a reviewer, an auditor, or a court — that the cross-platform interoperability claim is real and not aspirational.
3. **Standards-track readiness.** A canonical conformance corpus is a prerequisite for submission of the wire format to a standards body (IETF Independent Submission, W3C Community Group). By providing the corpus from day one, the invention is structurally prepared for standardisation in a way that ad-hoc implementations are not.

6.11 Worked Examples

This section provides four end-to-end worked examples that a PHOSITA can re-derive by hand or by trivial scripting from the algorithms specified above.

Example 1: Tiebreaker Resolution Consider a bundle of four hypervectors at $D = 8$. Suppose at bit position $i = 3$, exactly two of the four inputs have bit 3 set to 1 and two have it set to 0 — i.e., $\text{sum}[3] = 2 = n/2 = 4/2$, a tie. The tiebreaker is computed as:

```

domain  = "smritidb/tiebreak"           # 17 ASCII bytes
buf      = domain                       # 17 bytes
        || u32_le(8)                    # 0x08 0x00 0x00 0x00
        || u32_le(3)                    # 0x03 0x00 0x00 0x00
        || u32_le(4)                    # 0x04 0x00 0x00 0x00
                                           # total 29 bytes
digest   = BLAKE3(buf)                  # 32 bytes
out[3]   = digest[0] & 1                # least significant bit of the first

```

A PHOSITA can compute BLAKE3 of the 29-byte input using any reference BLAKE3 library, take the first byte of the result, and observe its low bit. That value is normatively out [3]. Every conformant implementation produces the identical value.

Example 2: Embedding Round-Trip Consider a 4-dimensional embedding $[0.3, -0.7, 0.0, 0.9]$ encoded at $D = 1024$. The quantised levels are (64, 15, 50, 94) as computed in §6.5.2. The encoder accumulates by XOR:

```

hv_0 = randomHv(BLAKE3("lv1:0:64"), 1024)
hv_1 = randomHv(BLAKE3("lv1:1:15"), 1024)
hv_2 = randomHv(BLAKE3("lv1:2:50"), 1024)
hv_3 = randomHv(BLAKE3("lv1:3:94"), 1024)
output = hv_0 XOR hv_1 XOR hv_2 XOR hv_3

```

Now consider a perturbed input $[0.305, -0.7, 0.0, 0.9]$. The first coordinate's quantised level is $\text{round}((1.305 * 99) / 2) = \text{round}(64.5975) = 65$ (the input being non-negative, both `Math.round` and `f32::round` give the same result, per §6.5.2.1). The perturbed encoding differs from the original only in the first summand, replacing `hv_0(level 64)` with `hv_0(level 65)`. Because `hv_0(64)` and `hv_0(65)` are independent random hypervectors, they disagree on approximately $D / 2 = 512$ of the 1024 bits. The XOR of these two hypervectors with the unchanged remainder produces an output that disagrees with the original in those approximately 512 positions, giving similarity ≈ 0.5 . For inputs that fall on the *same* quantised level (e.g., 0.3 and 0.301), the encoding is *bit-identical*. The encoder is therefore piecewise-constant in each coordinate, with similarity preservation occurring across multiple coordinates simultaneously.

Example 3: Consolidation Step Consider two hypervectors `a` and `b` at $D = 10000$ with Hamming distance 5000 (i.e., $\text{sim}(a, b) = 0.5$). Apply `pullCloser(a, b, maxSimDelta = 0.02, salt = 1)`.

```

disagree    = list of 5000 bit positions where a[i] != b[i]
maxFlips    = max(1, floor(0.02 * 10000)) = 200
toFlip      = min(5000, 200) = 200
seedBytes   = u64_le(1) = [01 00 00 00 00 00 00 00]
digestLen   = max(64, 200 * 4) = 800

```

```
digest      = BLAKE3-XOF([01 00 00 00 00 00 00 00], 800)
```

Sort the 5000 disagreeing bit positions by (digest[i mod 800], i) ascending, then flip the first 200: - Position disagree[0]: a[bit] := b[bit] (a moves toward b) - Position disagree[1]: b[bit] := a[bit] (b moves toward a) - Position disagree[2]: a[bit] := b[bit] - Position disagree[3]: b[bit] := a[bit] - ... and so on, alternating, for 200 positions total.

After the operation, the 200 selected disagreeing positions have been resolved to identical bits in a_out and b_out; the remaining 5000 - 200 = 4800 disagreeing positions are unchanged. The new Hamming distance between a_out and b_out is 4800, giving $\text{sim}(\text{a_out}, \text{b_out}) = 1 - 4800 / 10000 = 0.52$ — exactly 0.02 higher than the original. The Hamming distance between a_out and the original a is 100 (the 100 even-indexed positions where a was modified toward b), giving $\text{sim}(\text{a_out}, \text{a}) = 0.99$; symmetrically $\text{sim}(\text{b_out}, \text{b}) = 0.99$. Both items moved by exactly half the requested similarity delta, and the pair-similarity moved by the full delta.

Example 4: KMF Round-Trip Consider a substrate at D = 64 containing three items with arbitrary keys, payloads, and metadata. Serialise with writeKmf:

```
file = magic("KMF\0")                # 4 bytes   [0..4)
    || spec_version("0.1.0\0")        # 6 bytes   [4..10)
    || u64_le(header_offset)           # 8 bytes   [10..18)
    || hv_block                        # 3 * 8 = 24 bytes [18..42)
    || meta_block                      # JSON of length M [42..42+M)
    || value_block                     # length-prefixed payload
    || header_json                     # H = header_offset; then H+H
    || trailer("FMK\0")                # 4 bytes   [file_len-4..file_len)
```

The header is JSON of the form:

```
{
  "spec_version": "0.1.0",
  "dimension":    64,
  "item_count":   3,
  "created_at":   <unix-millis>,
}
```

```

    "index": [
      { "kind": "hv_block",      "offset": 18, "length": 24, "blake3": "<hex>" },
      { "kind": "meta_block",   "offset": 42, "length": <M>, "blake3": "<hex>" },
      { "kind": "value_block",  "offset": <42+M>, "length": <V>, "blake3": "<hex>" }
    ]
  }

```

To round-trip: a reader (a) verifies bytes 0–3 are "KMF\0", (b) verifies the last 4 bytes are "FMK\0", (c) reads the u64 at bytes 10–17 to locate the header, (d) JSON-decodes the header, (e) for each block in the header’s index slices the bytes at the listed offset/length and verifies the BLAKE3 digest, (f) deserialises each block into items. The resulting in-memory substrate is byte-equivalent (under the canonical equality of items) to the original. A re-write of the recovered substrate, with the same `created_at`, produces a file that is byte-identical to the original.

6.12 Industrial Applicability

The invention is industrially applicable across multiple computing domains in which an associative memory with deterministic cross-platform behaviour, low memory footprint, and verifiable persistence is required. Representative applications are:

(6.12.1) Large-language-model agent memory. An LLM-based agent maintains a persistent memory of past conversations, observations, and intermediate computations. The agent retrieves relevant memories on each turn by encoding the current context as a hypervector and recalling the top-k most similar items. The invention provides: (i) low memory footprint (a million memories at $D = 10000$ occupy approximately 1.25 GB, fitting comfortably in commodity RAM), (ii) deterministic recall across agent restarts and replicas, and (iii) auditability of memory evolution via the replayable consolidation procedure — a requirement in regulated agent deployments.

(6.12.2) Semantic search. Documents, code snippets, and structured records are encoded as hypervectors (whole-document hashes or aggregated chunk-level encodings) and recalled by similarity to a query. The invention’s bit-exact encoder removes the embedding-drift problem in which different versions of an embedding

model produce subtly different vectors for the same document, breaking ranking determinism across application versions. Because the encoders have no learned parameters, version-to-version reproducibility is exact.

(6.12.3) On-device retrieval-augmented generation (“on-device RAG”). A mobile or edge device runs an LLM and needs a local knowledge base to ground responses. Conventional vector databases (FAISS, hnswlib) require tens of megabytes of native code plus gigabytes of float-32 embedding storage; the invention runs the Rust binding (compiled to ARM64 mobile native code or to WebAssembly) with a small footprint (the WebAssembly build is expected to be less than 500 KiB pre-compression at SHA 17334f8; the exact measured byte size is to be confirmed before filing) and binary hypervector storage. The Kotlin (740) / Swift (750) bindings expose the same surface on Android and iOS respectively, with bit-exact behaviour identical to the server-side TypeScript (710) or Rust (720) implementation. This enables a single conformant memory to be replicated across cloud, server, and device tiers without translation.

(6.12.4) Federated and distributed memory. Multiple agents collaboratively maintain a shared associative memory. Each agent processes its own operation stream; periodically, the agents reconcile by exchanging KMF snapshots. Because every operation — `put`, `recall`, `consolidate` — is deterministic, two agents that process the same operations starting from the same snapshot converge to byte-identical state without any consensus protocol. The replayable consolidation procedure makes the reconciliation strictly an append-and-replay operation, not a stochastic merge.

(6.12.5) Verifiable audit trail. In regulated environments (healthcare assistants, legal-discovery agents, financial-compliance bots), the provenance of every memory item must be reconstructable. The invention’s `KMF snapshot at time t + operation log → state at time t` replay primitive provides exactly this: an auditor, given the snapshot and log, can re-derive the current state byte-for-byte and verify the agent’s behaviour at any point in its history.

(6.12.6) Cross-language data sharing in heterogeneous research pipelines. A research group’s data pipeline may involve a Python preprocessing stage, a Rust training stage, a TypeScript web visualisation stage, and a Kotlin / Swift mobile-

app evaluation stage. With conventional vector databases, sharing data across these stages requires format conversion at every boundary; with the invention, the same KMF byte stream is read directly by every stage, with cryptographic integrity guaranteed at the block level. This materially reduces the operational burden of multi-language research pipelines.

The combination of low memory footprint, deterministic cross-platform behaviour, verifiable persistence, and replayable consolidation has no direct equivalent in the prior art (Pinecone, Weaviate, FAISS, hnswlib, Annoy, pgvector, or in the academic HDC literature surveyed in the Background section). The invention is therefore not merely a “computer programme” but a system with concrete technical character, useful in industry, and capable of straightforward reproduction by a PHOSITA from the disclosure above.

7. CLAIMS

Explanatory Note on Unity of Invention and Described Embodiments

The claims that follow are unified by a single inventive concept, namely the **BLAKE3-anchored deterministic byte-identity of a binary hyperdimensional associative memory substrate**, by reason of which every primitive operation upon the said substrate — random hypervector generation, bundling, binding, permutation, thermometer encoding, text encoding, Hebbian consolidation, and wire-format serialisation — is reduced to a pure function of its inputs whose output is byte-for-byte identical across every conformant implementation, regardless of the processor architecture, programming language runtime, or memory layout in which the said implementation is executed. The independent claims are directed to six embodiments of this single concept, namely: (i) the deterministic majority-tiebreaker that anchors bundling, in method form (claim 1) and in system form (claim 7); (ii) the replayable Hebbian consolidation, bounded by a fixed similarity-drift parameter, that anchors associative reshape (claim 9); (iii) the open wire format with per-block BLAKE3 integrity that anchors persistence, in method form (claim 18) and in system form (claim 23); (iv) the thermometer-quantised cryptographically anchored projection encoder for bounded-range floating-point

input (claim 27); (v) the load-time conformance-gating apparatus for cross-implementation byte-identity (claim 31); and (vi) the permutation-positional text encoder built upon and cross-referencing the deterministic tiebreaker of claim 1 and the conformance corpus of claim 31 (claim 35). This concept is novel over the prior art at the priority date (BLAKE3 specification, O'Connor 2020; Kanerva 1988 sparse distributed memory; Rahimi, Kanerva and Rabaey 2016 ISLPED level encoder; Plate 1995 holographic reduced representations; Charikar 2002 sign-LSH/SimHash; Apache Parquet format; HDF5 Fletcher32; Merkle 1979 / Git / IPFS content-addressed storage) and is patentable as a unified invention under Section 16 of the Patents Act, 1970, the test for which is taken from *Genentech Inc.'s Patent* [1989] RPC 147 (UK, persuasive in India).

We Claim:

-
1. A computer-implemented method for producing a bit-identical binary hyperdimensional bundle hypervector across a plurality of heterogeneous language implementations executing on different processor architectures, the method comprising:
 - (a) receiving, by a processor, a multiset of n binary hypervectors each of fixed dimension D , wherein each said hypervector consists of bits in $\{0, 1\}$ and is represented as a packed bit-array of $\text{ceil}(D/8)$ bytes with most-significant-bit-first ordering within each byte;
 - (b) computing, for each bit position i in the range $[0, D)$, an integer sum s_i equal to the count of input hypervectors having bit value 1 at position i ;
 - (c) setting an output bit at position i to 1 if $s_i > n/2$ and to 0 if $s_i < n/2$; and
 - (d) for every bit position i at which s_i is exactly equal to $n/2$, deterministically resolving the tie by computing a BLAKE3 cryptographic hash digest over a byte string formed by concatenating a fixed domain-separation tag, the dimension D encoded as a 32-bit little-endian unsigned integer, the bit-index i encoded as a 32-bit little-endian unsigned integer, and the multiset multiplicity n encoded as a 32-bit little-endian unsigned integer, and setting the said output bit to the least-significant bit of the first byte of the said BLAKE3 digest;

characterised in that the said BLAKE3-derived tiebreaker bit, by operation upon the packed bit-array stored in the said memory, causes the said packed output hypervector to be byte-identical across heterogeneous processor architectures, thereby reducing storage input/output of the said associative memory substrate by permitting a single persisted snapshot to be loaded and verified bit-exactly by every conformant implementation without re-execution of the bundle operation; and wherein the said domain-separation tag, the said byte order, and the said field widths are recorded as a mandatory field of the wire-format header of any persisted snapshot of the said associative memory substrate, such that a foreign implementation reading the said snapshot is required to compute the said BLAKE3 digest by the identical byte layout in order to be admitted as conformant.

2. The method as claimed in claim 1, wherein the said fixed domain-separation tag is the ASCII byte string "`smritidb/tiebreak`", prepended to the said tuple before BLAKE3 hashing, so as to prevent collisions with BLAKE3 invocations made for other purposes by the same implementation.

3. The method as claimed in claim 1 or claim 2, wherein the said dimension D is a stored-substrate-wide constant selected from the group consisting of 1024, 8192, 10000, and 16384, and is fixed for the lifetime of the said associative memory substrate.

4. The method as claimed in any of claims 1 to 3, wherein each input hypervector of the said multiset is itself derived by expanding a 32-byte seed through a BLAKE3 extendable-output function to produce $\text{ceil}(D/8)$ bytes which are then unpacked most-significant-bit-first into a binary hypervector of dimension D .

5. The method as claimed in any of claims 1 to 3, further comprising emitting, alongside the said output hypervector, a BLAKE3 integrity hash computed over the packed bytes of the said output hypervector, the said integrity hash being persisted in a header index of a wire-format file to permit downstream verification of bundle output without re-execution.

6. The method as claimed in any of claims 1 to 3, wherein the said multiset of binary hypervectors represents a superposition of a plurality of role-filler bindings generated by a bind operation comprising element-wise exclusive-or of pairs of hypervectors, and the output hypervector produced by the said method is stored as an item key

in a content-addressable cleanup memory.

7. A computer system for maintaining a binary hyperdimensional associative memory substrate that is reproducible bit-exactly across heterogeneous processor architectures, thereby reducing storage input/output by permitting a single canonical snapshot to be shared between implementations without re-encoding, the system comprising at least one processor and a memory storing instructions which, when executed by the said processor, cause the system to:

- (a) maintain in the said memory a packed bit-array representation of a multiset of n binary hypervectors of fixed dimension D ;
- (b) compute a vertical sum across the said multiset for each of D bit positions;
- (c) write to an output packed bit-array a majority bit at each position at which the said sum is strictly greater than or strictly less than $n/2$; and
- (d) for each bit position at which the said sum equals $n/2$, invoke a deterministic tiebreaker module that returns the least-significant bit of the first byte of a BLAKE3 hash digest computed over the concatenation of a fixed ASCII domain-separation tag, the dimension D as a 32-bit little-endian unsigned integer, the said bit position as a 32-bit little-endian unsigned integer, and n as a 32-bit little-endian unsigned integer;

characterised in that the said memory comprises a non-volatile storage region holding the said packed bit-array between executions, and the said processor is operable to write to and read from the said non-volatile storage region, the said deterministic tiebreaker module being executed on the data path between the said memory and the said non-volatile storage region, such that the said output packed bit-array is byte-identical across every implementation of the said system regardless of processor word size, endianness in higher-level operations, or programming language runtime; and further characterised in that the said tiebreaker module is exposed via a foreign-function-interface boundary that is byte-transparent, such that an output produced by a native-code invocation thereof is byte-identical to an output produced by a WebAssembly-sandboxed invocation thereof on the same input multiset.

8. The system as claimed in claim 7, wherein the said at least one processor com-

prises a single-instruction-multiple-data vector unit, and wherein the said vertical sum is computed by a population-count operation over packed-bit lanes followed by horizontal accumulation, while the said deterministic tiebreaker module operates over a scalar control path so that the said output remains identical to a scalar reference implementation.

9. A computer-implemented method for incrementally reshaping a binary hyperdimensional associative memory substrate in response to access activity in a manner that is fully replayable from a wire-format snapshot together with an access log, thereby providing a holographic-fault-tolerant memory whose state can be reconstructed without retaining intermediate substrate copies, the method comprising:

- (a) maintaining, in a memory coupled to a processor, the said binary hyperdimensional associative memory substrate, a bounded sliding window of identifier batches recorded for successive recall operations against the said associative memory substrate, an access log recording, for each said recall operation, the identifier batch presented and a per-pass salt value associated with the said operation, and a co-activation tracker;
- (b) for each pair of identifiers (**a**, **b**) that co-occurs within any batch of the said sliding window, incrementing a pairwise co-activation counter $c(\mathbf{a}, \mathbf{b})$ of the said co-activation tracker, and decrementing the said counter when the corresponding batch is evicted from the said sliding window;
- (c) when the said co-activation counter $c(\mathbf{a}, \mathbf{b})$ reaches or exceeds a pull threshold, identifying a disagreement set comprising every bit position at which a binary key hypervector of item **a** stored in the said substrate differs from a binary key hypervector of item **b** stored in the said substrate;
- (d) bounding the number of bits to flip in the said disagreement set to a value computed as $\max(1, \text{floor}(\text{maxSimDelta} * D))$, where **maxSimDelta** is a bounded-similarity-drift parameter and **D** is the said dimension, and wherein $\text{floor}(\text{maxSimDelta} * D)$ is at most one percent of **D**;
- (e) deriving a deterministic ordering over the said disagreement set by computing a BLAKE3 hash digest over the said per-pass salt encoded as an 8-byte little-

endian unsigned integer, and ranking each said disagreeing bit position by a byte of the said digest indexed by the said bit position modulo the digest length, with ties broken by ascending bit position; and

- (f) flipping the said bounded number of bits in alternating direction, such that even-indexed flips set bits of the said binary key hypervector of item **a** to match the corresponding bit of the said binary key hypervector of item **b**, and odd-indexed flips set bits of the said binary key hypervector of item **b** to match the corresponding bit of the said binary key hypervector of item **a**, thereby moving the two said hypervectors equally and by no more than the said `maxSimDelta` in pairwise similarity per consolidation pass;

characterised in that each step of the said method is a pure function of the prior substrate state, the said access log, and the said per-pass salt, and the said access log is sufficient, together with a prior wire-format snapshot of the said substrate, to reconstruct the post-consolidation substrate state byte-for-byte by re-execution of the said method against the said snapshot and the said access log, without retention of any intermediate substrate copy; the said byte-identical post-consolidation substrate being persistable to a non-volatile storage device and re-instantiable on a different computing apparatus by re-execution of the said method against the said snapshot and access log, thereby dispensing with persistent storage of intermediate substrate states and reducing the storage footprint of the said associative memory.

10. The method as claimed in claim 9, wherein the said bounded-similarity-drift parameter `maxSimDelta` is a value not greater than 0.02, the said pull threshold is fixed at 32, and the said sliding window has a default size of 1000 recall batches.

11. The method as claimed in claim 9 or claim 10, further comprising flagging as cold any item whose last-accessed timestamp is older than a configured cold-age threshold and whose access count is below a configured minimum-access threshold, and persisting the said flag as part of the said substrate to permit subsequent bundled summarisation of cold items.

12. The method as claimed in any of claims 9 to 11, wherein the said deterministic ordering is computed by sorting the said disagreement set by ascending value of `digest[bit_index mod digest_length]`, with ties broken by ascending bit position, and wherein the digest is produced by a BLAKE3 extendable-output function

whose output length is at least $\max(64, 4 * \text{toFlip})$ bytes.

13. The method as claimed in any of claims 9 to 11, further comprising recording, for each consolidation pass, the said per-pass salt, the identifier pair acted upon, and the count of bits flipped, into the said access log, so that a verifier can replay the said method against an earlier said snapshot and produce a byte-identical substrate.

14. The method as claimed in any of claims 9 to 11, wherein the said sliding window is maintained as an append-and-trim queue of identifier batches, and pairwise co-activation counters are decremented when their last contributing batch is evicted, so that the memory footprint of the said co-activation tracker is bounded by the said window size.

15. The method as claimed in any of claims 9 to 11, further comprising a non-volatile storage device storing the said snapshot together with the said access log, and a restore module configured to replay the said access log against the said snapshot to reconstruct an equivalent substrate state on a second computing apparatus.

16. The method as claimed in any of claims 9 to 11, wherein the said memory further stores a typed configuration record exposing the said `maxSimDelta`, the said pull threshold, the said sliding-window size, a cold-age threshold, and a cold-minimum-access threshold as separately tunable parameters.

17. The method as claimed in any of claims 9 to 11, wherein the said pairwise co-activation counters are evicted from the said memory when their value reaches zero, so that the working-set memory footprint of the said co-activation tracker scales with the active co-activation graph rather than the cardinality of the said substrate.

18. A computer-implemented method for serialising and deserialising a binary hyperdimensional associative memory substrate to and from a single byte-addressed wire-format container that is streaming-friendly and verifiable, thereby providing BLAKE3-backed data integrity and reduced storage input/output for an implementation-independent associative memory substrate, the method comprising:

- (a) writing, at the start of an output byte sequence, a fixed magic byte string "KMF\0" and a six-byte ASCII specification-version field;
- (b) reserving an eight-byte little-endian header-offset field immediately following

the said specification-version field;

- (c) writing one or more data blocks, the said data blocks comprising n entries where n is an item count of the said substrate, each said data block being of a kind selected from the group consisting of (i) a hypervector block comprising n packed binary hypervectors each of $\text{ceil}(D/8)$ bytes laid out in column-major order with most-significant-bit-first packing within each byte, (ii) a metadata block comprising n rows of item metadata, and (iii) a length-prefixed value block comprising n opaque payloads each preceded by a 32-bit little-endian unsigned length;
- (d) computing a BLAKE3 hash digest over the said bytes of each said data block, encoding the said digest as a hexadecimal string, and recording the kind, the byte offset, the byte length, and the said hexadecimal hash for each said data block in an index;
- (e) writing, after all the said data blocks, a header object containing the said specification-version, a conformance-corpus version identifier referencing the canonical conformance corpus against which the said substrate has been verified, the said dimension D , the said item count n , a created-at timestamp, and the said index;
- (f) writing, after the said header object, a fixed trailer magic byte string "FMK\0"; and
- (g) populating the previously reserved said header-offset field with the byte offset of the said header object as a 64-bit little-endian unsigned integer;

characterised in that a reader of the said byte sequence verifies the said trailer magic before trusting the said header-offset, verifies each said data block against its recorded BLAKE3 hash before deserialising the said data block, and rejects any said byte sequence whose said specification-version field is unsupported, so that corruption, truncation, or version drift is detected before substrate state is reconstructed; and further characterised in that (i) the said hypervector block is laid out with each said packed binary hypervector occupying exactly $\text{ceil}(D/8)$ bytes with no per-vector padding, the said dimension D being constrained to be a multiple of 8, such that a partial reader is operable to compute the byte offset of the k -th hypervector within the said block as $k * \text{ceil}(D/8)$ bytes without consulting any per-vector in-

dex; and (ii) the said trailer magic byte string "FMK\0" differs from the said header magic byte string "KMF\0" by reversal of its first three bytes, providing a directional integrity marker distinguishable from accidental file concatenation; and (iii) the said reader is operable to read the said byte sequence from a non-volatile storage device into a memory of a computing apparatus, and to refuse to write any substrate state into a working memory of the said apparatus until the said per-block BLAKE3 hash digests have been verified.

19. The method as claimed in claim 18, wherein the said header object is serialised as a JSON document and the said metadata block is encoded as a JSON document, the said header object being positioned after the said data blocks rather than before them, so as to permit the said data blocks to be streamed and per-block BLAKE3-hashed in a single forward pass without prior knowledge of the said data-block byte lengths and without requiring a seek-to-front rewrite of the said header.

20. The method as claimed in claim 18 or claim 19, wherein the said wire-format container is configurable to compress each said data block with the Zstandard algorithm at a selectable compression level prior to the said BLAKE3 hash being computed, and wherein the said header object records, per block, the compression algorithm and parameters used.

21. The method as claimed in any of claims 18 to 20, wherein the said reader, upon encountering a specification-version field of a different major version than that supported by the said reader, refuses to deserialise the said byte sequence and surfaces a typed `UnsupportedSpecVersion` error condition.

22. The method as claimed in any of claims 18 to 20, further comprising emitting an optional attic block recording one or more bundled summaries of cold items together with their source-item identifiers, the said attic block being independently hashed and indexed in the said index.

23. A computer system for persisting a binary hyperdimensional associative memory substrate to a verifiable open wire format, thereby providing reduced storage input/output through column-major partial loading and BLAKE3-backed integrity verification, the system comprising at least one processor, a memory storing instructions, and a non-volatile storage device coupled to the said processor and selected

from the group consisting of a local file system, an IndexedDB object store within a web-browser, a SQLite database row, and an object-storage bucket; the said instructions, when executed by the said processor, causing the system to:

- (a) emit a fixed magic byte string and a specification-version field to an output sink hosted by the said non-volatile storage device, followed by a reserved eight-byte header-offset field;
- (b) emit one or more data blocks each comprising a contiguous sequence of bytes of a kind selected from a hypervector block, a metadata block, a length-prefixed value block, and an attic block;
- (c) compute a BLAKE3 hash digest over the bytes of each said data block as the said bytes are emitted;
- (d) emit, after the said data blocks, a header object recording the said specification-version, a dimension D, an item count, a created-at timestamp, and an index of (kind, offset, length, BLAKE3 hash) tuples;
- (e) emit a fixed trailer magic byte string after the said header object; and
- (f) backfill the said reserved header-offset field with the byte offset of the said header object;

characterised in that the said header object is written after the said data blocks so as to permit single-pass streaming, the said system is operable to produce a byte-identical wire-format byte sequence regardless of which of the said non-volatile storage devices is selected as the output sink, and a reader module of the said system verifies the said trailer magic, the said specification-version, and the said per-block BLAKE3 hash digests before reconstructing any substrate state into a working memory of the said apparatus.

24. The system as claimed in claim 23, further comprising a Zstandard compressor coupled between a block-emit pipeline and the said BLAKE3 hasher, the said compressor being configurable per block kind, and the said header object recording per-block compression parameters.

25. The system as claimed in claim 23 or claim 24, further comprising a partial-load module configured to seek directly to a hypervector block of the said wire format and demand-load a contiguous slice thereof without requiring deserialisation of the

said metadata block or the said value block.

26. The system as claimed in any of claims 23 to 25, wherein the said reader module, upon detecting a mismatch between a computed BLAKE3 hash digest and a digest recorded in the said header object, surfaces a typed `CorruptSnapshot` error condition and aborts reconstruction prior to returning any substrate item to a caller.

27. A computer-implemented method for mapping an input bounded-range floating-point vector of length N to a binary hypervector of fixed dimension D such that pairwise cosine similarity in the said input space is approximately preserved as Hamming similarity in the said output space, thereby providing a deterministic, training-free, lower-compute-load embedding pipeline that requires no neural-network inference, the method comprising:

- (a) receiving an input vector $\mathbf{v}$ of N floating-point values, each clamped to the range $[-1, 1]$;
- (b) initialising an accumulator hypervector of dimension D to all zeros;
- (c) for each index i in $[0, N)$:
 - (i) computing a thermometer-quantised level $lv1_i$ as a function of the said clamped value $\mathbf{v}[i]$ and a fixed levels constant L , specifically $lv1_i = \text{clamp}(\text{round}((\mathbf{v}[i] + 1) * (L - 1) / 2), 0, L - 1)$;
 - (ii) constructing a per-coordinate seed byte string by concatenating a fixed domain-separation tag "lv1:", the decimal ASCII representation of i in canonical decimal ASCII form without leading zeros or sign characters, the ASCII byte ':', and the decimal ASCII representation of the said level $lv1_i$ in the same canonical decimal ASCII form;
 - (iii) computing a BLAKE3 hash digest of the said seed byte string;
 - (iv) expanding the said digest by a BLAKE3 extendable-output function to $\text{ceil}(D/8)$ bytes and unpacking the said bytes most-significant-bit-first into a per-coordinate binary hypervector of dimension D ; and
 - (v) updating the said accumulator hypervector by a bind operation comprising element-wise exclusive-or with the said per-coordinate hypervector;

and

(d) returning the said accumulator hypervector as the said binary hypervector; characterised in that the said method binds a per-coordinate random projection that depends jointly on the coordinate index and the thermometer-quantised level, the said projection being derived from a BLAKE3 extendable-output function applied to the said canonical decimal ASCII seed byte string rather than from a sliding correlation between two random seed hypervectors or from an implementation-defined pseudo-random number generator, so that two said input vectors which are close in cosine similarity produce per-coordinate hypervectors that agree on the same set of bits, and consequently the said exclusive-or accumulators agree on a number of bits proportional to the said cosine similarity, without requiring any trained model, gradient descent, or floating-point matrix multiplication in the said output path; and further characterised in that an output produced by a first implementation of the said method on a first processor architecture is byte-identical to an output produced by a second implementation of the said method on a second, different processor architecture, the said byte-identity being verifiable by computing the SHA-256 digest of the packed bytes of the said accumulator hypervector and comparing the said digest, byte-for-byte, against a value recorded in a canonical conformance corpus shipped with each said implementation; and yet further characterised in that the said method is executed entirely in integer arithmetic and bitwise operations on the said processor, without invocation of any floating-point matrix-multiplication unit or neural-network inference accelerator, such that the said binary hypervector is producible on a microcontroller-class processor lacking floating-point hardware.

28. The method as claimed in claim 27, wherein the said levels constant L is a value of at least 64 and not greater than 256, and the said fixed domain-separation tag is the ASCII byte string "lvl:".

29. The method as claimed in claim 27 or claim 28, wherein the said dimension D is selected from the group consisting of 1024, 4096, 8192, 10000, and 16384.

30. The method as claimed in any of claims 27 to 29, further comprising storing the said binary hypervector as a key in a content-addressable cleanup memory and retrieving, in response to a noisy cue hypervector, the top-k nearest stored hypervectors by Hamming similarity.

31. A computer system for verifying cross-implementation byte-identity of a binary hyperdimensional associative memory substrate, thereby providing a load-time technical guard against the admission of a non-conformant substrate to the said memory, the system comprising at least one processor, a non-volatile storage device storing a canonical conformance corpus file encoded as a JSON document, and a memory storing instructions which, when executed by the said processor, cause the system to:

- (a) maintain, in the said non-volatile storage device, the said canonical conformance corpus comprising a plurality of test groups, each said test group specifying (i) a canonical input including a textual seed, a numeric dimension, and where applicable an operation type, and (ii) an expected output expressed as a SHA-256 hexadecimal digest computed over the packed bytes of the canonical output hypervector or a numeric expected scalar with a stated tolerance;
- (b) execute, against the said memory, each said test group by deriving an actual output hypervector or scalar from the said canonical input using substrate primitives of an implementation under test;
- (c) compute, in the said processor, the SHA-256 hexadecimal digest of the packed bytes of the said actual output hypervector and compare the said digest, byte-for-byte, against the said expected digest in the said corpus; and
- (d) report the said implementation as conformant only if the said comparison succeeds for every said test group of bit-exact kind and the said actual scalar lies within the said stated tolerance of the said expected scalar for every said test group of approximate kind;

characterised in that the said system rejects, at load time of any substrate snapshot produced by a foreign implementation, any said snapshot whose computed output, when subjected to each said test group, fails to match the said expected digest or to lie within the said stated tolerance, the said system thereby refusing to write the rejected said snapshot into a working memory of the said apparatus and providing a load-time technical guard against admission of a non-conformant substrate to the said associative memory; and further characterised in that the said canonical conformance corpus comprises both (i) bit-exact test groups whose expected output

is a SHA-256 digest of packed bytes, and (ii) approximate test groups whose expected output is a numeric scalar paired with a stated absolute tolerance, the combination of both kinds in a single corpus being adapted to verify an approximate-vector data structure for which only certain primitive operations are required to be bit-exact while other operations are required only to be within tolerance.

32. The system as claimed in claim 31, wherein the said test groups comprise at minimum a random-hypervector group, a string-encoding group, a similarity-pairs group, a bind-round-trip group, and a bundle group, each said group being keyed by the same canonical inputs in every said implementation; and, where text encoding is supported by the implementation, a bag-of-words text-encoding group and a character-n-gram text-encoding group.

33. The system as claimed in claim 31 or claim 32, wherein the said canonical conformance corpus further specifies, in addition to the said SHA-256 digest, the said dimension and the said seed encoding as UTF-8, so that no implementation is required to make an undocumented choice that could perturb the said digest.

34. The system as claimed in any of claims 31 to 33, wherein the said canonical conformance corpus is generated by a designated reference implementation and is reviewed and frozen prior to release, and wherein any subsequent change to the said corpus requires a corresponding bump of a specification-version field.

35. A computer-implemented method for mapping a textual input to a binary hypervector of fixed dimension D in a manner that preserves word order and is deterministic, training-free, and free of any neural-network inference, thereby providing a lower-compute-load and improved-memory-footprint alternative to learned text embeddings for use as an associative-memory key, the method comprising:

- (a) normalising the said textual input by, where configured, lowercasing all characters and replacing every non-alphanumeric character with a whitespace, then splitting the said normalised text on whitespace into a sequence of words, and discarding any word shorter than a configured minimum word length;
- (b) for each window of n consecutive words ($w_0, w_1, \dots, w_{\{n-1\}}$) over the said sequence of words, computing an n -gram hypervector by:

- (i) deterministically encoding each said word w_j to a per-word hypervector for w_j of dimension D by computing a BLAKE3 hash digest of the byte string formed by concatenating a fixed string-domain tag "word:" with the UTF-8 bytes of w_j and expanding the said digest by a BLAKE3 extendable-output function to $\text{ceil}(D/8)$ bytes unpacked most-significant-bit-first;
- (ii) leaving the said per-word hypervector for the first word w_0 of the window unrotated, and for each subsequent word w_j with j in $[1, n)$, applying a cyclic permutation of the said per-word hypervector for w_j by exactly j positions, the said cyclic permutation being defined as the element-wise mapping that places the element at position p of an input vector at position $(p + j) \bmod D$ of the output vector; and
- (iii) combining the said unrotated first per-word hypervector with the $n - 1$ permuted subsequent per-word hypervectors by a bind operation comprising element-wise exclusive-or to produce the said n -gram hypervector; and
- (c) bundling the said n -gram hypervectors across all the said windows by element-wise majority with deterministic tiebreaker resolution as claimed in claim 1, and returning the said bundle as the said binary hypervector encoding the said textual input;

characterised in that the use of a position-dependent cyclic permutation, defined as the element-wise mapping that places the element at position p of an input vector at position $(p + j) \bmod D$ of the output vector for a window-position offset j , ensures that two textual inputs differing only in word order produce different n -gram hypervectors and consequently different bundle outputs, thereby distinguishing the said method from a bag-of-features binary fingerprint construction in which window position is not encoded; and the said method does so without any tokeniser vocabulary, any trained word embedding, or any floating-point arithmetic on the hot path; and further characterised in that the output of the said method is byte-identical across heterogeneous implementations verified by the canonical conformance corpus of the system of claim 31; the said method being computable entirely by integer arithmetic and bitwise operations, such that the said binary hypervector is producible on

a processor lacking any floating-point unit, and is storable in a non-volatile storage device as an associative-memory key without further encoding.

36. The method as claimed in claim 35, wherein the said configured minimum word length is 3 and the said window size n is 2 or 3; and further wherein, when the said sequence of words has length less than n , the method falls back to encoding the said normalised text as a bag-of-words bundle of per-word hypervectors without permutation; and further comprising a character- n -gram variant in which the said sequence of words is replaced by a sequence of overlapping character n -grams drawn from the said normalised text, and the said per-shingle hypervector is derived from the BLAKE3 hash digest of a fixed character-domain tag "char:" concatenated with the said shingle, thereby providing tolerance to single-character typographical errors.

8. ABSTRACT

A computer-implemented system and method for a binary-hypervector associative memory store characterised by deterministic cross-implementation byte-identical state. Conventional vector databases suffer from silent numerical drift across language runtimes and substantial memory footprint owing to floating-point representations. The disclosed system addresses these problems by: a deterministic tiebreaker for bundle operations using a cryptographic hash over a domain-separation tag, dimension, bit index and multiplicity; a packed-bit wire format with per-block hash integrity and a directional magic trailer; a replayable Hebbian consolidation procedure with bounded similarity drift; a thermometer-quantised random projection encoder for bounded floating-point inputs; a load-time conformance-gating apparatus admitting only byte-identical implementations; and a permutation-positional n -gram text encoder. The combination yields bit-exact interoperability between native, managed and browser runtimes, an approximately thirty-two-fold reduction in memory footprint relative to single-precision vectors, and verifiable fault tolerance through corruption detection. Reference is invited to Fig. 1 of the accompanying drawings.

Signature of Inventor: _____ Vivek Singh

Date: _____ **Place:** _____

Signature of Applicant (for Tanvrit Private Limited): _____

Designation: _____ **Date:** _____

End of Complete Specification. Pinned to source SHA 17334f8.